

Informatik 2

Kapitel 8 – Fensterklassen implementieren

Inhaltsverzeichnis

8.1 Einleitung	2
8.2 Grafische Komponenten und ihr Layout	5
8.2.1 Fallstudie: ClickFrame (Fensterdarstellung)	5
8.2.2 Grafische Komponenten in der Java-API	8
8.2.3 Grafisch darstellbare Objekte (Component)	9
8.2.4 Grafische Container (Container)	12
8.2.5 Fensterklassen (Window)	15
8.2.6 Leichtgewichtige Fensterkomponenten (JComponent)	17
8.3 Ereignisbehandlung	18
8.3.1 Fallstudie: ClickFrame (Ereignisbehandlung)	18
8.3.2 Behandlung von Benutzerinteraktionen	24
8.3.3 Ereignisarten	25
8.4 Fenster und Dialoge zu Datenklassen	33
8.4.1 Fallstudie: Angestelltenverwaltung	33
8.4.2 Grundregeln und Ausblicke	42
8.4.3 Standarddialoge	44
8.5 Zeichnen in Fenstern	45
8.5.1 Fallstudie: Mausposition zeichnen	45
8.5.2 Allgemeines Konzept	47
8.5.3 Fallstudie: Grafischer Editor	51
8.6 Weitere Fallstudien	61

8.1 Einleitung

In diesem Kapitel wollen wir lernen, wie wir Programme entwerfen und implementieren, mit denen der Nutzer über eine grafische Benutzeroberfläche (englisch *Graphical User Interface*, GUI) interagiert. Ein solche grafische Benutzeroberfläche besteht in der Regel aus einem Hauptanwendungsfenster und gegebenenfalls mehreren untergeordneten Dialogfenstern für bestimmte Funktionalitäten der Anwendung.

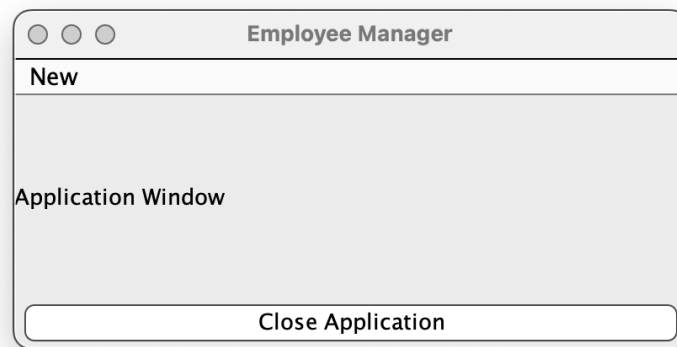


Abbildung 1: Screenshot des Hauptanwendungsfensters der Angestelltenverwaltung.

Für die im letzten Kapitel begonnene Angestelltenverwaltung könnte das Hauptanwendungsfenster zum Beispiel wie in Abbildung 1 aussehen. Wir sehen oben den Titel `EmployeeManager`. Im unteren Bereich (man sagt: **im Süden**) hat das Fenster eine Schaltfläche (englisch: **Button**) zum Beenden der Anwendung. In der Mitte (man sagt: **im Zentrum**) befindet sich eine Beschriftung (englisch: **Label**), die anzeigt, dass es sich um das Anwendungsfenster handelt.

Zur Programmierung eines solchen Fensters bietet Java zum Beispiel die Klasse `JFrame`. Ein `JFrame`-Objekt ist ein Fenster mit einem Titel und einem Layout, das verschiedene grafische Komponenten enthalten kann und diese nach dem Layout anordnet. In diesem Beispiel enthält es ein `JLabel`-Objekt für die Beschriftung und ein `JButton`-Objekt für die Schaltfläche. Diese sind hier nach dem sogenannten **BorderLayout** angeordnet, nach dem das Fenster in einen südlichen, nördlichen, westlichen, östlichen und zentralen Bereich aufgeteilt ist. Hierbei können Bereiche leer bleiben (wie in der Abbildung der nördliche, westliche und östliche Bereich).

Links oben ist ein Menü des Fensters sichtbar (New). Ein solches Menü kann ausgeklappt werden und klickbare Einträge oder Untermenü enthalten. In unserem Beispiel wollen wir über dieses Menü untergeordnete Dialoge zum Beispiel zur Anzeige einer Liste aller Angestellten (**Listendialog**) oder zum Erfassen eines neuen Angestellten (**Erfassungsdialog**) öffnen können. Ein solcher Erfassungsdialog könnte wie in Abbildung 2 aussehen. Ein Dialog ist ebenfalls ein Fenster.

Der Erfassungsdialog hat den Titel `Register New Employee`. Im Süden hat der Dialog verschiedene von links nach rechts angeordnete Buttons (`Ok`, `Apply`, `Cancel`) für unterschiedliche Benutzeraktionen. Eine solche Anordnung erhält man durch das sogenannte **FlowLayout**. Dazu enthält der Dialog im Süden ein Panel (Klasse `JPanel`). Das ist eine grafische Komponente mit einem Layout, die andere grafische Komponenten enthalten kann. Ein Panel hat keine eigene grafische Darstellung und dient nur als ein sogenannter **Container** für andere grafische Komponenten. Im

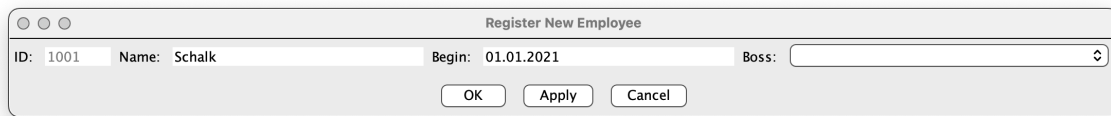


Abbildung 2: Screenshot eines Erfassungsfensters für Angestellte.

Beispiel enthält das Panel die Buttons und ordnet diese nach dem FlowLayout an. Durch die Benutzung solcher Panels lässt sich die Anordnung der grafischen Elemente in einem Fenster hierarchisch gliedern. Im oberen Bereich (man sagt: **im Norden**) befindet sich ebenso ein Panel. Dieses enthält von links nach rechts nach dem FlowLayout angeordnet vier Beschriftungen, drei Textfelder (Klasse `TextField`) und ganz rechts eine Drop-Down-Auswahl (Klasse `ComboBox`). Über die Textfelder und die Drop-Down-Auswahl können die Daten eines neuen Angestellten eingegeben werden. Das erste Textfeld ganz links für die ID ist dabei ausgegraut und es ist kann nichts eingegeben werden, da es nicht aktiviert ist. Das liegt daran, dass die IDs automatisch vergeben und voreingetragen werden.

Die Grafik in Abbildung 3 zeigt weitere mögliche Fenster und Dialoge der Anwendung und ihr Zusammenspiel durch Benutzeraktionen als sogenannter **Zustandsautomat**. In einer Anwendung können grundsätzlich mehrere Fenster (und Dialoge) gleichzeitig geöffnet sein. Allerdings ist immer nur eines dieser geöffneten Fenster aktiv und nur im aktiven Fenster sind Benutzeraktionen möglich.

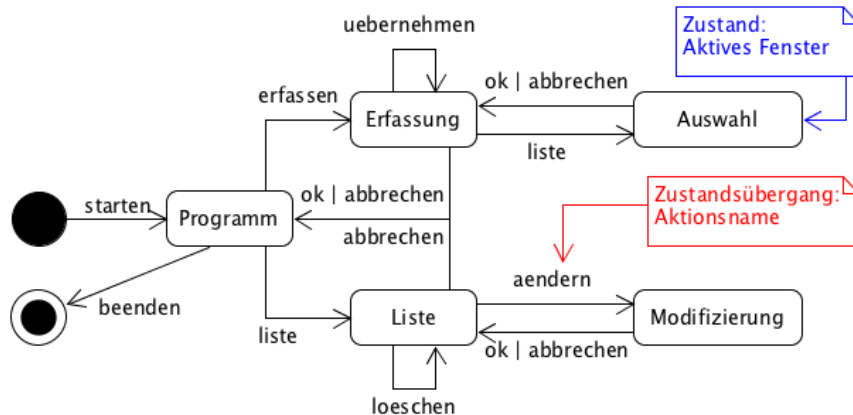


Abbildung 3: Übersicht über verschiedene Fenster einer grafischen Benutzerschnittstelle.

Die Knoten in diesem Automaten repräsentieren jeweils ein aktives Fenster. Die von einem Knoten ausgehenden Pfeile repräsentieren Benutzeraktionen, die im jeweiligen Fenster (zum Beispiel über Buttons) möglich sind. Die Pfeile führen zum Fenster, welches nach der Benutzeraktion aktiv ist.

Mit Programmstart landen wir ausgehend vom großen schwarzen Punkt auf der linken Seite (dem Startknoten) beim Hauptanwendungsfenster (oder **Programmfenster**). Der Benutzer kann hier ein Fenster zum Erfassen eines neuen Angestellten öffnen (erfassen), oder ein Fenster mit einer Liste aller Angestellten öffnen (liste) oder das Programm beenden (beenden, dann landet man beim Endknoten links unten).

Im Listenfenster wiederum kann man

- mit abbrechen zum Programmfenster zurückkehren
- mit loeschen einen aus der angezeigten Liste ausgewählten Angestellten löschen
- mit aendern für einen aus der angezeigten Liste ausgewählten Angestellten ein Modifizierungsfenster öffnen um einzelne Daten zu ändern.

Ausgehend vom Modifizierungsfenster kann man

- mit abbrechen zum Listenfenster zurückkehren, ohne Datenänderungen zu übernehmen
- mit ok zum Listenfenster zurückkehren, wobei die Datenänderungen übernommen werden

Ausgehend vom Erfassungsfenster lässt sich

- mit abbrechen zum Programmfenster zurückkehren, ohne einen neuen Angestellten anzulegen
- mit ok zum Programmfenster zurückkehren, wobei ein neuer Angestellter mit den eingegebenen Daten angelegt wird
- mit uebernehmen ein neuer Angestellter anlegen
- mit liste ein Auswahlfenster öffnen, um von dort Daten eines vorhandenen Angestellten für das Anlegen eines neuen Angestellten zu übertragen

Ausgehend vom Auswahlfenster lässt sich

- mit abbrechen zum Erfassungsfenster zurückkehren, ohne Daten zu übernehmen
- mit ok zum Erfassungsfenster zurückkehren, wobei die Daten eines aus der angezeigten Liste ausgewählten Angestellten übernommen werden

Diese Übersicht ist rein exemplarisch zu verstehen. Sie werden zwar die konzeptuellen Grundlagen zur Umsetzung solcher Fenster kennenlernen, wir werden eine solche GUI aber nicht komplett durchprogrammieren. Erstens wäre das im Rahmen der Vorlesung zu aufwändig, denn man landet schnell bei einer 4-stelligen Anzahl von Code-Zeilen. Zweitens ist die genaue Ausgestaltung einer GUI von Anwendung zu Anwendung an unterschiedliche Kriterien geknüpft und deren Diskussion ist nicht Gegenstand dieser Vorlesung.

Als weiteres Beispiel wollen wir in diesem Kapitel eine GUI für den grafischen Editor betrachten. Diese GUI unterscheidet sich grundsätzlich von der GUI der Angestelltenverwaltung. Beim grafischen Editor werden die Daten nämlich nicht über Textfelder (und ähnliche grafische Komponenten wie zum Beispiel Auswahlfelder) eingegeben, sondern über eine Zeichenfläche. Zeichenflächen haben eine gesonderte konzeptuelle Grundlage und betrachten wir erst im Unterkapitel 8.5.

Im folgenden Unterkapitel 8.2 werden wir uns zuerst nur um die **Grundbausteine und das Layout von Fenstern** kümmern. In den weiteren Unterkapiteln lernen wir dann, wie wir ein Fenster

dazu bringen, auf Benutzeraktionen zu reagieren (das nennt man **Ereignisbehandlung**, Unterkapitel 8.3), und wie wir die Fenster einer Anwendung **mit den Datenklassen aus der Fachkonzeptschicht verbinden** (Unterkapitel 8.4). Im letzten Unterkapitel 8.5 lernen wir schließlich noch **Zeichenflächen** kennen.

8.2 Grafische Komponenten und ihr Layout

Möchte man in Java ein Fenster implementieren, so macht man das grundsätzlich **als Unterklasse einer API-Fensterklasse**. Dadurch stehen einem alle Grundfunktionalitäten von Fenstern aus der API zur Verfügung. Gleichzeitig kapselt man die Komponenten eines Fensters mit deren Layout und Ereignisbehandlung übersichtlich wartbar in einer Klasse. Außerdem macht man das Fenster wiederverwendbar.

8.2.1 Fallstudie: ClickFrame (Fensterdarstellung)

Als kleines Fallbeispiel wollen wir ein Fenster ClickFrame mit einem Label Count clicks: neben einem Button mit der Beschriftung 0 im Süden erstellen. Später wollen wir zählen, wie oft der Benutzer auf den Button klickt, und die Anzahl immer in der Beschriftung des Buttons anzeigen.

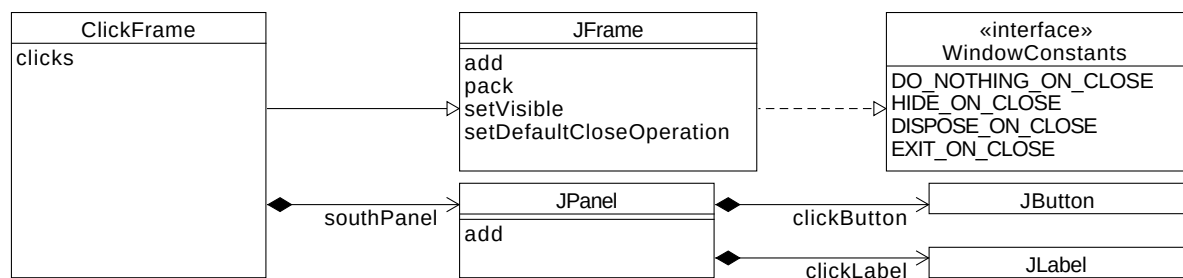


Abbildung 4: UML-Entwurfsmodell für ein Fenster ClickFrame.

Als erstes erstellen wir in Abbildung 4 ein UML-Diagramm dafür. Wir erstellen unser Fenster also als Unterklasse von JFrame. Dadurch können wir nützliche JFrame-Methoden benutzen. Für unser Beispiel benötigen wir add, pack, setVisible und setDefaultCloseOperation.

Gemäß Diagramm wird ein ClickFrame-Objekt aus einem JPanel-Objekt bestehen, welches wiederum aus einem JLabel- und einem JButton-Objekt besteht.

Das Fenster hat ein Attribut clicks, mit dem wir speichern werden, wie oft der Benutzer auf den Button klickt.

Die Implementierung in Java kann ohne Ereignisbehandlung wie folgt aussehen:

```

import javax.swing.JFrame;
import javax.swing.JPanel;
import javax.swing.JButton;
import javax.swing.JLabel;
import java.awt.BorderLayout;

public class ClickFrame extends JFrame {

    private int clicks = 0;

    public ClickFrame() {
        super("ClickFrame");

        JPanel southPanel = new JPanel();
        JButton clickButton = new JButton(clicks + "");

        southPanel.add(new JLabel("Count clicks:"));
        southPanel.add(clickButton);

        this.add(southPanel, BorderLayout.SOUTH);

        this.setDefaultCloseOperation(EXIT_ON_CLOSE);

        pack();
        setVisible(true);
    }
}

```

Im Konstruktor wird zuerst mit `super("ClickFrame")` ein Konstruktor der Oberklasse `JFrame` aufgerufen. Als Parameter übergeben wir den Titel des Fensters. Danach werden alle Komponenten des Fensters angelegt und hinzugefügt:

- `JPanel southPanel = new JPanel()`
Erzeugt ein neues Panel `southPanel`. Dieses dient als Behälter für die weiteren Komponenten. Ein Panel hat als Standard das `FlowLayout`. Seine Komponenten werden automatisch von links nach rechts angeordnet in der Reihenfolge, in der sie hinzugefügt werden.
- `JButton clickButton = new JButton(clicks + "")`
Erzeugt einen neuen Button `clickButton`. Dessen Beschriftung übergeben wir an den Konstruktor als Parameter.
- `southPanel.add(new JLabel("Count clicks:"))`
Fügt dem Panel `southPanel` ein Label mit der Beschriftung `Count clicks:` hinzu. Dieses erzeugen wir durch einen anonymen Konstruktoraufruf (das heißt, ohne eine Referenz auf das Label anzulegen) und übergeben es als Parameter an die `add`-Methode der Klasse `JPanel`.
- `southPanel.add(clickButton)`
Fügt danach dem Panel `southPanel` den Button `clickButton` hinzu.

- `this.add(southPanel, BorderLayout.SOUTH)`
Fügt dem Fenster im Süden das Panel `southPanel` hinzu. Ein Fenster hat als Standard das BorderLayout. Dieses teilt das Fenster in fünf Bereiche, in die wir jeweils eine Komponente hinzufügen können. Hier übergeben wir dazu an die `add`-Methode der Klasse `JFrame` die Komponente `southPanel` und den Bereich `BorderLayout.SOUTH`. Dazu liegen alle Bereiche als Klassenkonstanten der Klasse `BorderLayout` vor.
- `this.setDefaultCloseOperation(EXIT_ON_CLOSE)`
Legt die gewünschte Reaktion fest, wenn der Benutzer auf das Fensterkreuz klickt. Als Parameter der Methode stehen die folgenden Klassenkonstanten der Schnittstelle `WindowConstants` zur Verfügung (diese wird von `JFrame` und `JDialog` implementiert):

DO_NOTHING_ON_CLOSE	Keine Reaktion, Fenster bleibt offen
EXIT_ON_CLOSE	Programm wird beendet (gleiche Wirkung wie <code>System.exit(0)</code>)
HIDE_ON_CLOSE	Fenster wird unsichtbar gemacht (gleiche Wirkung wie <code>setVisible(false)</code>)
DISPOSE_ON_CLOSE	Fenster schließen, aber Programm weiterlaufen lassen. Wenn es sich um das letzte Fenster handelt, das geschlossen wird, wird das Programm beendet.

- `pack()`
Legt die optimale Größe des Fensters fest, damit alle enthaltenen Komponenten dargestellt werden können. Alternativ kann man mit der `setSize`-Methode der Klasse `JFrame` auch eine feste Fenstergröße setzen. Man sollte `pack` immer erst aufrufen, wenn alle Komponenten hinzugefügt wurden.
- `setVisible(true)`
Macht das Fenster sichtbar. Das ist notwendig, da Fenster standardmäßig unsichtbar sind. Sollte immer als letztes aufgerufen werden.

Will man ein solches Fenster nun erzeugen, so ruft man in `main` einfach seinen Konstruktor auf:

```
public class ClickProgram {
    public static void main (String args[]) {
        new ClickFrame();
    }
}
```

Hier genügt ein anonymer Konstruktoraufwurf. Das Fenster wird erst wieder gelöscht, wenn das Programm beendet wird, oder wir das selbst über den Aufruf der `dispose`-Methode der Klasse `JFrame` machen.

Übung: Sie sollten nun den Code einmal abtippen und versuchen, das Fenster erfolgreich zu erstellen. Variieren Sie dann zum Beispiel den Titel, sowie Anordnung, Aussehen und Anzahl der Komponenten, um die Auswirkungen verschiedener API-Methoden auf das tatsächliche Aussehen des Fensters zu beobachten.

8.2.2 Grafische Komponenten in der Java-API

Nach diesem Auftaktbeispiel wollen wir uns nun einen systematischen Überblick über die Java-API für die Erstellung grafischer Komponenten verschaffen. In der Java-Klassenbibliothek gibt es dafür mehrere Pakete, von welchen wir folgende Pakete näher betrachten werden:

- `java.awt`
Benötigen wir für das Layout von Fenstern.
- `javax.swing`
Aus diesem Paket kommen unsere Fensterkomponenten.
- `java.awt.event`
Hier finden wir alles zur bereits angesprochenen Ereignisbehandlung, also die Reaktion auf Benutzeraktionen.

Wie das auch in anderen Bereichen der Fall ist, bilden die grafischen Komponenten wieder eine Vererbungshierarchie. In Abbildung 5 sehen Sie einen ersten wichtigen Ausschnitt:

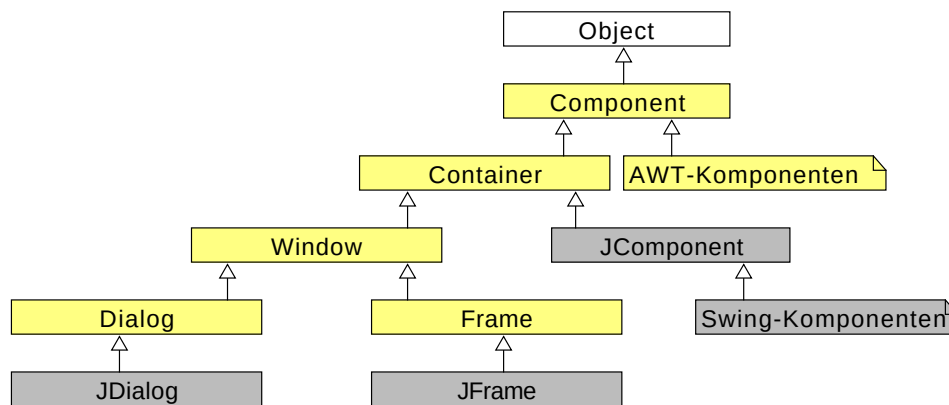


Abbildung 5: Vererbungshierarchie grafischer Komponenten (Ausschnitt).

Die beiden anders dargestellten Knoten AWT-Komponenten und Swing-Komponenten repräsentieren keine Klassen, sondern weisen auf weitere Unterklassen hin, deren Darstellung an dieser Stelle zu unübersichtlich wäre (diese sehen wir uns später in separaten Grafiken an). Alle **gelb** hinterlegten Klassen gehören zum Paket `java.awt`. AWT steht für das sogenannte **Abstract Window Toolkit**. Dieses war die erste API, die in Java für grafische Benutzerschnittstellen mitgeliefert wurde. Es nutzt sogenannte **Peer-Klassen**. Diese bezeichnen *Partner-Klassen des Betriebssystems* mit der eigentlichen Implementierung, die hinter den Java-Klassen stehen, mit denen wir arbeiten. AWT-Klassen setzen die grafischen Elemente somit nicht selbst um, sondern rufen Methoden des Betriebssystems auf. Dies führt einerseits dazu, dass die Darstellung der grafischen Komponenten in starkem Maß von der jeweiligen Plattform abhängt. Andererseits gleichen die AWT-Komponenten im Erscheinungsbild (**look and feel**) den Standardkomponenten der jeweiligen Plattform. Da Java betriebssystemunabhängig funktionieren soll, sind in AWT nur jene grafischen Elemente enthalten, die in allen unterstützten Betriebssystemen vorkommen. Alle AWT-Komponenten erfordern intern Aufrufe an das Betriebssystem, da sie von diesem bereitgestellt werden. Somit nutzen sie

Speicherverwaltung außerhalb von Java und Kommunikation mit dem Betriebssystem. Da sie somit stark vom Betriebssystem abhängen und dessen Ressourcen verwenden, spricht man bei AWT-Komponenten von sogenannten **schwergewichtigen Komponenten**. AWT bietet eine gute Performanz und gute Portabilität, aber Handhabung und Design sind veraltet.

Definition 8.1: Schwergewichtige Komponenten

Eine **schwergewichtige Komponente** ist eine grafische Komponente, die nicht vollständig von Java selbst gezeichnet wird, sondern eine entsprechende grafische Komponente des Betriebssystems nutzt.

Alle grafischen Komponenten im Paket `java.awt` (**AWT-Komponenten**) sowie Fensterklassen im Paket `javax.swing` (**Swing-Komponenten**) sind schwergewichtig.

Die **grau** hinterlegten Klassen gehören zum Paket `javax.swing`. Java bietet dieses Paket als modernere Alternative zu AWT an. Swing-Komponenten bezeichnet man als **leichtgewichtige Komponenten**, da sie nicht vom Betriebssystem erstellt werden müssen. Stattdessen sind die Komponenten größtenteils plattformunabhängig in Java implementiert, sodass ein plattformübergreifend einheitliches look-and-feel möglich ist. Ein paar Komponenten basieren dennoch auf AWT-Klassen und sind dadurch auf das Betriebssystem angewiesen. Dies betrifft insbesondere die Fensterklassen `JFrame` und `JDialog`, da Fenster nicht ohne das Betriebssystem funktionieren können. Was in den Fenstern passiert, ist dann aber wiederum Sache von Java. Swing bietet viele zusätzliche Komponenten, erweiterte AWT-Komponenten und 2D-Unterstützung für die einfachere Entwicklung komplexer Oberflächen. Medienunterstützung, Animationen oder 3D-Darstellung sind allerdings nicht möglich (dafür gibt es andere Alternativen wie zum Beispiel JavaFX).

Definition 8.2: Leichtgewichtige Komponenten

Eine **leichtgewichtige Komponente** ist eine grafische Komponente, die vollständig von Java selbst gezeichnet wird. Leichtgewichtige Komponenten sind typischerweise Teil einer übergeordneten schwergewichtigen Komponente wie einem Fenster.

Alle Swing-Komponenten, die keine Fenster sind, sind leichtgewichtig.

8.2.3 Grafisch darstellbare Objekte (Component)

Die gemeinsame Oberklasse für alle **grafischen Komponenten**, mit denen der Benutzer per Maus oder Tastatur interagieren kann (sowohl für AWT als auch für Swing) ist die Klasse `Component`. Sie stellt viele nützliche gemeinsame Funktionalitäten bereit, unter anderem:

- das Setzen der Position (`setLocation`)
- das Setzen der Größe (`setSize`)
- das Setzen von Schriftart und -größe (`setFont`)
- das Setzen von Hintergrund- und Vordergrundfarbe (`setBackground`, `setForeground`)

- das Aktivieren / Deaktivieren (`setEnabled`)
- das Sichtbarmachen / Unsichtbarmachen (`setVisible`)

Position und Größe einer Komponente

Position und Größe werden dabei jeweils in Anzahl von Bildschirm-Pixeln angegeben. Mit

```
myComponent.setSize(100, 200);
```

macht man `myComponent` zum Beispiel 100 Pixel breit und 200 Pixel hoch. Die Positionen werden relativ zu einem Bezugspunkt festgelegt. Bei einem Fenster ist dieser Bezugspunkt die linke obere Bildschirmcke, bei einer Komponente eines Fensters die linke obere Ecke dieses Fensters. Mit

```
myComponent.setLocation(100, 200);
```

wird die linke obere Ecke von `myComponent` ausgehend vom Bezugspunkt um 100 Pixel nach rechts und 200 Pixel nach unten verschoben. Experimentieren Sie mit verschiedenen Werten, um ein Gefühl dafür zu bekommen.

Definition 8.3: Position einer Komponente

Die **Position** einer Komponente wird gemessen durch den horizontalen und vertikalen Abstand ihrer linken oberen Ecke zu einem übergeordneten Bezugspunkt.

Der Bezugspunkt bei Fenstern ist die linke obere Ecke des Bildschirms. Ist eine Komponente Teil einer übergeordneten Komponente, so ist der Bezugspunkt die linke obere Ecke der übergeordneten Komponente.

Abstände werden in Pixeln angegeben.

Farbe einer Komponente

Farben (für Hinter- und Vordergrund von Komponenten) werden in Java über `Color`-Objekte verwaltet. Bei deren Erzeugung wird das sogenannte **RGB-Farbmodell** benutzt, das grundsätzlich für alle selbst leuchtenden Geräte wie Bildschirme zum Einsatz kommt (im Gegensatz zum Beispiel zu Leinwänden). Dieses Modell beruht auf der Dreifarben-theorie, wonach alle Farben aus drei Primärfarben gemischt werden können. Jede Farbe entsteht durch **additive Mischung** der **Rot**-, **Grün**- und **Blau**werte. Additiv bedeutet, dass die Mischfarben durch Überlagerung der Primärfarben entstehen. Je nach **Stärke** der drei Primärfarben, die miteinander gemischt werden, entsteht eine andere Farbe. Wird Rot und Grün in gleicher Stärke gemischt, entsteht zum Beispiel ein Gelbton. Der Abbildung 6 können weitere Mischungen entnommen werden. Mischt man alle Grundfarben in gleicher Stärke, so entsteht ein Grauton. Je höher bei den Mischungen die Stärke der Farben ist, umso heller wird das Ergebnis. So entsteht die Farbe Weiß durch Mischung der Primärfarben in jeweils maximaler Stärke. Schwarz ist hingegen das Ausbleiben aller Primärfarben.

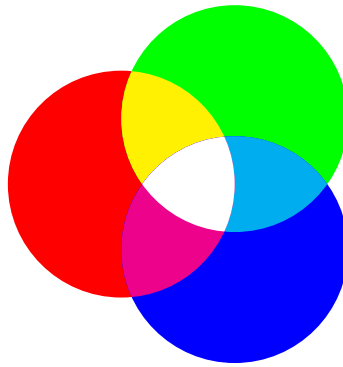


Abbildung 6: Additive Farbmischung.

Wenn Ihnen das ungewohnt vorkommt, so liegt das daran, dass sich Farben auf nicht selbst leuchtenden Hintergründen wie Leinwänden subtraktiv mischen (durch die unterschiedliche Absorption des auf den Hintergrund fallenden Lichts). Vielleicht erinnern Sie sich aber auch noch an den Physik-Unterricht in der Schule: dort lernt man, dass weißes Sonnenlicht aus einer Überlagerung (Mischung) der Regenbogenfarben besteht und durch Prismen in diese zerlegt werden kann.

In Java wird eine Farbe mit dem Konstruktor

```
public Color(int r, int g, int b)
```

erzeugt. Hierbei bedeuten **r**, **g** und **b** die Stärken der Farben rot, grün und blau in der Mischung. Diese Stärken nennt man **Farbwerte**. Diese Farbwerte liegen zwischen 0 und 255. Experimentieren Sie mit diesem Wissen ein wenig mit der Erzeugung bestimmter Farben wie weiß, hellgrau, dunkelgrau, schwarz und so weiter. Die Klasse **Color** stellt außerdem zahlreiche Klassenkonstanten wie **Color.RED** zur Verwendung vordefinierter Farben zur Verfügung.

Definition 8.4: Farbe einer Komponente

Alle **Farben** werden aus den **Primärfarben** rot, grün und blau gemischt. Jede Primärfarbe kann dabei einen **Farbwert** zwischen 0 und 255 haben. Farben lassen sich durch die Klasse `java.awt.Color` erzeugen.

Schrift einer Komponente

Schriften werden von der Klasse **Font** repräsentiert. Eine Schrift wird erzeugt mit dem Konstruktor

```
public Font(String name, int style, int size)
```

Jede Schrift wird festgelegt durch

- einen Namen `name`:
Hier stehen feste Schriftnamen wie zum Beispiel `Arial`, `Times`, `Courier` oder `Helvetica` zur Verfügung. Einige Schriftnamen stehen auch als Klassenkonstanten zur Verfügung.
- einen Schriftstil `style`:
Hier stehen feste Schriftstile wie zum Beispiel „**fett**“ (repräsentiert durch die Klassenkonstante `Font.BOLD`), „*kursiv*“ (`Font.ITALIC`) oder „normal“ (`Font.PLAIN`) zur Verfügung.
- eine Schriftgröße `size`:
Diese wird als ganze Zahl in der Maßeinheit *Punkt* angegeben.

8.2.4 Grafische Container (Container)

Die Klasse `Container` ist direkte Unterklasse von `Component` und gemeinsame Oberklasse für alle grafisch darstellbaren Objekte, die andere grafisch darstellbare Objekte enthalten können. Ein `Container`-Objekt ist also ein **Behälter für andere grafische Komponenten**. Der Abbildung 5 lässt sich entnehmen, dass es weitere AWT-Komponenten gibt, die keinen solchen Behälter sind. Andererseits sind insbesondere alle Fenster und auch Swing-Komponenten Behälter. Die Klasse `Container` stellt unter anderem folgende Funktionalitäten bereit:

- Hinzufügen von Komponenten zum Behälter (`add`)
- Anordnen von im Behälter enthaltenen Komponenten (`setLayout`)

Die Komponenten, die einem Behälter hinzugefügt werden, werden in einer (internen) Liste verwaltet. Die Anordnung der Komponenten in einem Behälter wird durch das festgelegte Layout bestimmt. Dieses wird intern durch ein `LayoutManager`-Objekt verwaltet. Jeder Behälter hat dabei ein Standardlayout. Das Standardlayout von Fenstern ist das BorderLayout (Klasse `BorderLayout`) und das eines Panels das Flowlayout (Klasse `FlowLayout`). Möchte man das Layout eines Behälters wechseln, so übergibt man dafür das passende `LayoutManager`-Objekt an die `setLayout`-Methode. Zum Beispiel bekommt ein Panel `borderLayoutPanel` durch folgende Anweisung

```
borderLayoutPanel . setLayout ( new BorderLayout ( ) ) ;
```

ein BorderLayout. In dieser Vorlesung besprechen wir exemplarisch das BorderLayout, das Flowlayout und das Gridlayout (Klasse `GridLayout`).

BorderLayout

Das BorderLayout ist in fünf Bereiche aufgeteilt: Norden, Süden, Osten, Westen und Zentrum. Jeder Bereich enthält höchstens eine Komponente. Sollte man mehr als 5 Komponenten platzieren wollen, kann man wie im Eingangsbeispiel ein Panel einsetzen, das weitere Komponenten aufnehmen kann. Enthält ein Bereich keine Komponente, so wird er nicht dargestellt. Die Größe eines Bereichs richtet sich nach der Größe der enthaltenen Komponente. Einem Behälter `borderLayoutContainer` mit BorderLayout kann wie folgt eine neue grafische Komponente `newComponent` im Bereich `part` hinzugefügt werden:

```
borderLayoutContainer . add ( newComponent , part ) ;
```

Der Bereich `part` wird dabei durch eine der folgende Klassenkonstanten angegeben:

- **Norden:** `BorderLayout.NORTH`
- **Süden:** `BorderLayout.SOUTH`
- **Osten:** `BorderLayout.EAST`
- **Westen:** `BorderLayout.WEST`
- **Zentrum:** `BorderLayout.CENTER`

Abbildung 7 zeigt ein Fenster mit `BorderLayout`. In jedem der 5 Bereiche befindet sich ein Label mit jeweils unterschiedlicher Hintergrundfarbe.

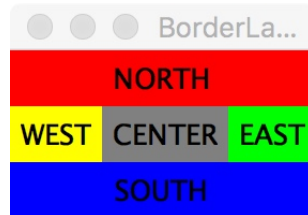


Abbildung 7: Screenshot eines Fensters mit `BorderLayout`.

Übung: Versuchen Sie dieses Fenster nun zur Übung selbst zu implementieren (alle dafür notwendigen Konzepte und Methoden wurden bereits besprochen).

FlowLayout

In einem `FlowLayout` können beliebig viele Komponenten von links nach rechts hinzugefügt werden. Ändert man die Komponentenorientierung des Containers, sind auch Anordnungen von rechts nach links möglich. Einem Behälter `FlowLayoutContainer` mit `FlowLayout` kann wie folgt eine neue grafische Komponente `newComponent` rechts angefügt werden:

```
FlowLayoutContainer.add(newComponent);
```

Abbildung 8 zeigt ein Fenster mit `FlowLayout`. Es enthält von links nach rechts vier Labels mit jeweils unterschiedlicher Hintergrundfarbe und mit unterschiedlicher Schriftart und -größe.

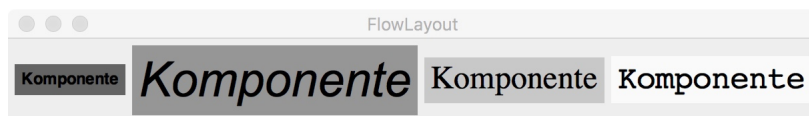


Abbildung 8: Screenshot eines Fensters mit `FlowLayout`.

Übung: Versuchen Sie auch dieses Fenster zur Übung selbst zu implementieren (alle dafür notwendigen Konzepte und Methoden wurden wieder bereits besprochen).

GridLayout

Das GridLayout ordnet die Komponenten in einer Tabelle mit jeweils gleich großen Zellen an. Jede Zelle kann eine Komponente beinhalten. Die Anzahlen der Zeilen und Spalten der Tabelle werden dem Konstruktor übergeben, zum Beispiel:

- `new GridLayout(n, m)`
Erzeugt ein GridLayout mit `n` Zeilen und `m` Spalten.
- `new GridLayout(n, 0)`
Erzeugt ein GridLayout mit `n` Zeilen und einer dynamischen Anzahl von Spalten (hängt von der Anzahl der Komponenten ab).
- `new GridLayout(0, m)`
Erzeugt ein GridLayout mit `m` Spalten und einer dynamischen Anzahl von Zeilen (hängt von der Anzahl der Komponenten ab).

Einem Behälter `GridLayoutContainer` mit GridLayout kann wie folgt eine neue grafische Komponente `newComponent` angefügt werden:

```
GridLayoutContainer.add(newComponent);
```

Dabei wird `newComponent` entweder rechts in der aktuellen Zeile angefügt (falls die Zeile noch nicht voll ist) oder als erstes Element der nächsten Zeile.

Abbildung 9 zeigt ein Fenster mit GridLayout. Es enthält sechs Labels in zwei Zeilen zu je drei Spalten mit jeweils unterschiedlicher Hintergrundfarbe.



Abbildung 9: Screenshot eines Fensters mit GridLayout.

Übung: Nun versuchen Sie bitte wieder auch dieses Fenster zur Übung selbst zu implementieren.

Hierarchisch gegliedertes Layout

Wie bereits im Eingangsbeispiel beschrieben, kann man Layouts durch die Verwendung von Panels hierarchisch gliedern.

Übung: Überlegen Sie, wie man zum Beispiel das Fenster in Abbildung 10 implementieren könnte.

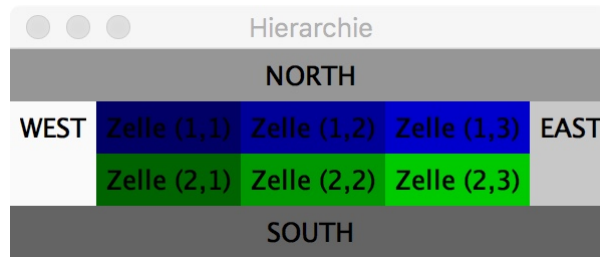


Abbildung 10: Screenshot eines Fensters mit hierarchisch gegliedertem Layout.

8.2.5 Fensterklassen (Window)

Eine direkte Unterklasse von `Container` ist die Klasse `Window`. Sie stellt die gemeinsame Oberklasse für alle Fenster dar mit Methoden zum

- Schließen (`dispose`)
- Setzen der optimalen Größe zur Darstellung aller enthaltener Komponenten (`pack`)

Leichtgewichtige Komponenten müssen immer in einer schwergewichtigen Hauptfensterklasse enthalten sein. Daher muss zum Programmieren eines Fensters immer eine schwergewichtige Hauptfensterklasse verwendet werden. Swing stellt unter anderem die schwergewichtigen Hauptfensterklassen `JFrame` und `JDialog` zur Verfügung.

Die Klasse `JFrame` repräsentiert Fenster mit Rand und Titel und wird für Hauptanwendungsfenster (oder **Primärdialoge**) benutzt. Ein **Primärdialog** wird zur direkten Aufgabenerfüllung benutzt, zum Beispiel als Editor. Der Titel kann im Konstruktor gesetzt werden.

Die Klasse `JDialog` repräsentiert Fenster mit Rand und Titel für modale oder nicht-modale untergeordnete Dialoge (oder **Sekundärdialoge**), die immer zu einem übergeordneten Primärdialog gehören. Ein **Sekundärdialog** wird für situationsabhängige, zusätzliche Informationen / Aktionen genutzt, zum Beispiel zum *Drucken*, *Speichern unter* oder *Suchen und Ersetzen*. Ein Sekundärdialog wird aus einem Primärdialog (dem **Elterndialog**) gestartet, welcher nach Beendigung des Sekundärdialogs fortgesetzt wird. Titel, Elterndialog und Modalität werden im Konstruktor gesetzt.

Ist ein Sekundärdialog **modal**, so blockiert er andere Fenster der Anwendung und muss daher vor Aktivierung eines anderen Fensters beendet werden. Ein **nicht-modaler** Sekundärdialog kann dagegen von anderen Fenstern unterbrochen werden. Ein Sekundärdialog wird mit dem Primärdialog geschlossen.

Definition 8.5: Dialoge

Ein **Primärdialog** ist das Hauptfenster einer Anwendung und bleibt in der Regel während der gesamten Nutzung der Anwendung geöffnet. Eine geeignete Klasse für Primärdialoge ist `JFrame`.

Ein **Sekundärdialog** ist ein Zusatzfenster einer Anwendung und wird in der Regel nur temporär benötigt. Ein Sekundärdialog kann modal oder nicht-modal sein. Eine geeignete Klasse für Sekundärdialoge ist `JDialog`.

Ein **modaler Sekundärdialog** blockiert alle anderen Dialoge der Anwendung, solange er geöffnet ist.

Ein **nicht-modaler Sekundärdialog** kann von anderen Dialogen der Anwendung unterbrochen werden.

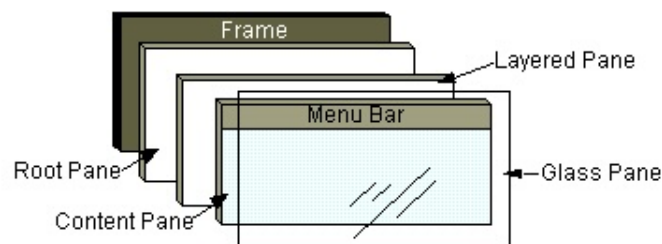


Abbildung 11: Aufbau eines schwergewichtigen Swing-Fensters (von <https://docs.oracle.com/javase/tutorial/uiswing/components/rootpane.html>).

Die Swing-Hauptfensterklassen sind in mehrere **Scheiben (Panels)** gegliedert (siehe Abbildung 11):

- Der **Rootpane** (Klasse `RootPane`) enthält die weiteren Scheiben und befindet sich im Hintergrund.
- Davor liegt der **Layeredpane** (Klasse `LayeredPane`). Im Layeredpane können Komponenten überlappend in verschiedenen Schichten (Layer) angeordnet werden.
- Eine der Schichten enthält den **Contentpane** (Klasse `ContentPane`). Dieser enthält die (optionale) Menüleiste und alle Komponenten, die über eine `add`-Methode des Fensters bezüglich einem Layout hinzugefügt wurden. Alle Komponenten im Contentpane sind nebeneinander (nicht überlappend) angeordnet.
- Der **Glasspane** (Klasse `GlassPane`) liegt über allen anderen Scheiben und ist standardmäßig durchsichtig und unsichtbar. Auf ihm kann man zum Beispiel komponentenübergreifend zeichnen.

Leichtgewichtige Komponenten können zu jeder dieser Scheiben hinzugefügt werden. Der Contentpane wird standardmäßig zur Definition der GUI benutzt und in allen bisherigen Beispielen haben wir nur den ContentPane benutzt. Die weiteren Scheiben werden wir in dieser Vorlesung nicht verwenden.

8.2.6 Leichtgewichtige Fensterkomponenten (JComponent)

Schließlich wollen wir noch die Klasse `JComponent` besprechen. Diese stellt die Oberklasse für alle **leichtgewichtigen GUI-Komponenten** dar. Diese können nur innerhalb eines (schwergewichtigen) Hauptanwendungsfensters angezeigt werden. Eine gemeinsame Funktionalität für alle leichtgewichtigen Komponenten ist die folgende Methode:

- `setOpaque`: Damit kann die Komponente **durchsichtig** oder **undurchsichtig** gemacht werden.

Ist sie undurchsichtig, so wird jeder Pixel der Komponente gezeichnet. Ist sie durchsichtig, so werden darunterliegende Pixel (der dahinter liegenden Komponenten) angezeigt. Konkrete Unterklassen haben diesbezüglich unterschiedliche Standardeinstellungen. Abbildung 12 zeigt einen Ausschnitt der Vererbungshierarchie der Unterklassen von `JComponent`. Diese Komponenten werden wir in den Beispiel- und Übungsprogrammen dieser Vorlesung benutzen. Hier finden Sie Kurzbeschreibungen zur Orientierung:

- `JMenuBar`: Leiste, um Menüs aufzunehmen.
- `JMenu`: Menü mit Titel, das Menüpunkte enthalten kann.
- `JMenuItem`: Menüpunkt mit Titel, um Aktionen auszuführen, Optionen zu setzen oder andere Menüs (zum Ausklappen) aufzunehmen.
- `JButton`: Schaltfläche mit Beschriftung oder Icon (Symbol) für Benutzeraktionen.
- `JLabel`: Komponente zur Anzeige von Text oder einem Icon.
- `JPanel`: Komponente zur Aufnahme weiterer Komponenten.
- `TextField`: Komponente zur Eingabe von Text (einzeilig).
- `TextArea`: Komponente zur Eingabe von Text (mehrzeilig).
- `FileChooser`: Dateiauswahldialog.
- `ComboBox`: Auswahl aus einer Drop-Down-Liste.
- `OptionPane`: Standarddialog zur Benutzereingaben oder Benutzerbenachrichtigungen (kann zum Beispiel für Warndialoge eingesetzt werden).

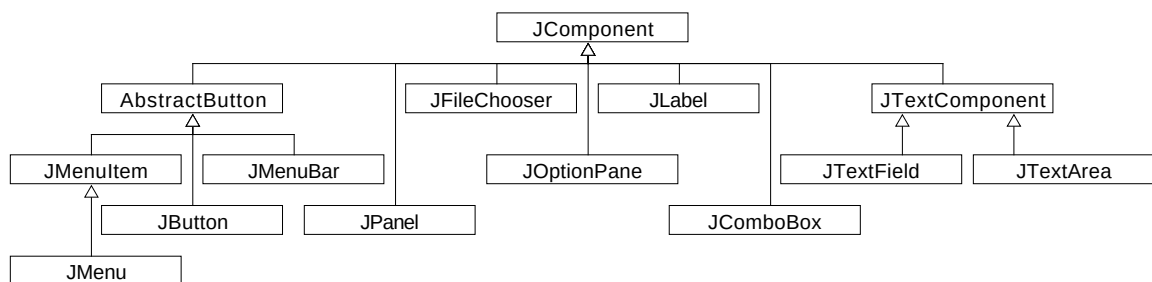


Abbildung 12: Überblick über leichtgewichtige Swing-Komponenten

Übung: Platzieren Sie die verschiedenen Komponenten versuchsweise selbst in einem Fenster, damit Sie wissen, wie sie aussehen.

8.3 Ereignisbehandlung

Noch reagieren unsere Fenster nicht auf Benutzeraktionen. Zum Beispiel gibt es bisher keine Reaktion auf Drücken von Schaltflächen, Aktivieren von Checkboxes, Drücken von Maus- oder Tastaturtasten, Mausbewegungen, und so weiter. Dazu werden zuerst unser Beispielfenster `ClickFrame` um eine Ereignisbehandlung erweitern und dann allgemein alle existierenden Ereignisbehandlungsarten besprechen.

8.3.1 Fallstudie: ClickFrame (Ereignisbehandlung)

Wir wollen jetzt unser Beispielfenster `ClickFrame` aus dem letzten Unterkapitel 8.2 erweitern, so dass es wie folgt auf Benutzeraktionen reagiert:

Klickt der Benutzer auf den Button `clickButton`, so soll die auf dem Button angezeigte Zahl um eins erhöht werden (wir wollen so die Anzahl der Button-Klicks zählen)

Für die Implementierung existieren viele verschiedene Varianten, von denen wir nun 5 Varianten vorstellen. Sie sollten grundsätzlich alle 5 Varianten beherrschen.

Variante 1: Innerer Abhörer mit Lambda-Ausdruck

Die folgende Implementierung tut das Gewünschte:

```
public class ClickFrame extends JFrame {

    private int clicks = 0;

    public ClickFrame() {
        super("ClickFrame");

        JPanel southPanel = new JPanel();
        JButton clickButton = new JButton(clicks + "");

        southPanel.add(new JLabel("Count clicks:"));
        southPanel.add(clickButton);

        this.add(southPanel, BorderLayout.SOUTH);

        this.setDefaultCloseOperation(EXIT_ON_CLOSE);

        clickButton.addActionListener(e -> {
            ++clicks;
            clickButton.setText(clicks + "");
        });

        pack();
        setVisible(true);
    }
}
```

Um auf den Button-Klick reagieren zu können, müssen wir beim Button `clickButton` mit der `addActionListener`-Methode einen speziellen **Ereignisabhörer**, einen sogenannten **Actionlistener**, registrieren. Ein Actionlistener ist ein Objekt, das die Schnittstelle `ActionListener` implementiert. Diese Schnittstelle enthält die abstrakte `actionPerformed`-Methode. Deren Implementierung legt die Reaktion auf den Button-Klick fest. Da die `ActionListener`-Schnittstelle eine funktionale Schnittstelle ist, können wir die Implementierung der `actionPerformed`-Methode als Lambda-Ausdruck übergeben:

- Eingabe für die `actionPerformed`-Methode ist ein **Actionevent e** (welches durch den Button-Click ausgelöst wird).
- Im Rumpf der `actionPerformed`-Methode wird das Attribut `clicks` inkrementiert und dann mit `setText` die Beschriftung des Buttons `clickButton` neu gesetzt.

In Java funktioniert die Reaktion auf Benutzerinteraktionen mit grafischen Komponenten allgemein so:

- Die Interaktion mit einer grafischen Komponente löst ein **Ereignis** aus
- Dieses Ereignis hören wir über einen **Ereignisabhörer** ab
- Den Ereignisabhörer müssen wir dazu bei der grafischen Komponente registrieren
- Die grafische Komponente nennen wir die **Ereignisquelle**.
- Der Ereignisabhörer enthält den Code, den wir als Reaktion auf die Interaktion ausführen wollen

In unserem Beispiel ist der Button die Ereignisquelle, und der Lambda-Ausdruck definiert den Ereignisabhörer. Im UML-Diagramm stellt sich das wie in Abbildung 13 dar. Hierbei sind API-Klassen in blau und eigene Klassen in rot dargestellt.

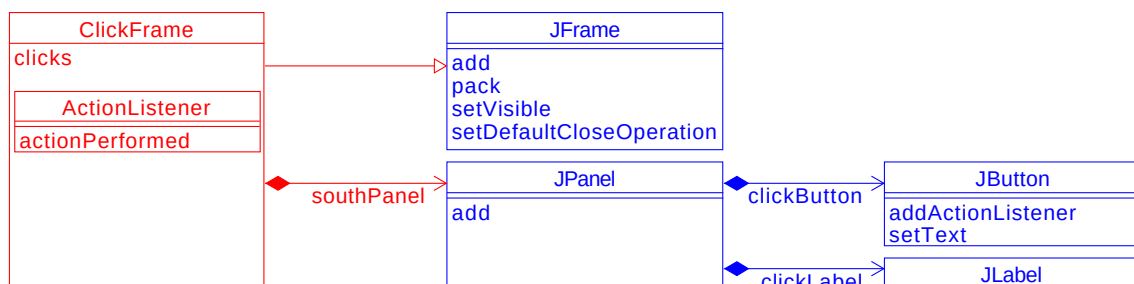


Abbildung 13: UML-Entwurfsmodell für ein Fenster `ClickFrame` mit einem inneren Ereignisabhörer.

Der Lambda-Ausdruck erzeugt ein `ActionListener`-Objekt, das durch eine sogenannte **innere Klasse** von `ClickFrame` definiert ist. Eine innere Klasse ist eine Klasse, die nur innerhalb einer anderen (äußeren) Klasse definiert ist, und deshalb auch nur dort zur Verfügung steht. In UML wird eine innere Klasse innerhalb der äußeren Klasse gezeichnet.

Übung: Implementieren und testen sie diese Variante.

Variante 2: Unterklasse von JButton mit innerem Abhörer

Zu Variante 1 kann man feststellen, dass dort zwar die Information über die Anzahl der Button-Klicks im Fenster (über das Attribut `clicks`) verwaltet wird, das Fenster selbst aber gar nichts mit der Darstellung dieser Information zu tun hat. Vielmehr übernimmt der Button die Darstellung. Das legt nahe, auch die Information (also das Attribut) im Button selbst zu verwalten. Um das zu erreichen, implementieren wie in Abbildung 14 eine eigene Unterklasse `ClickButton` der API-Klasse `JButton`.

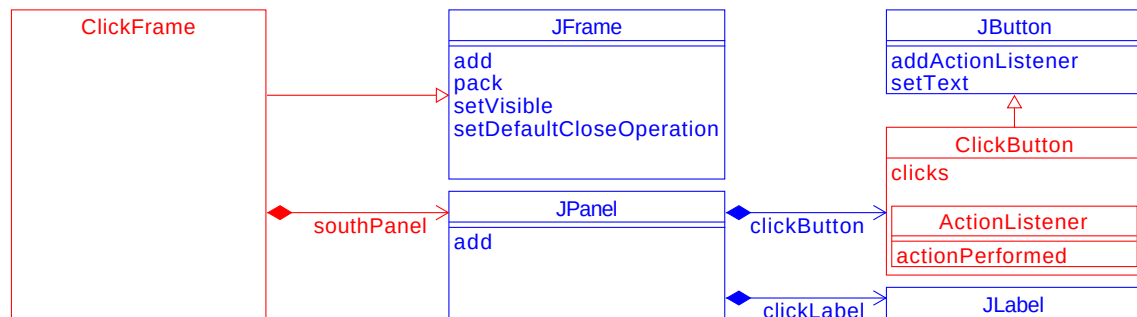


Abbildung 14: UML-Entwurfsmodell für ein Fenster `ClickFrame` mit einer Unterklasse von `JButton` mit innerem Ereignisabhörer.

Den inneren Ereignisabhörer können wir auch in der neuen Unterklasse `ClickButton` durch einen Lambda-Ausdruck erzeugen:

```
public class ClickButton extends JButton {

    private int clicks = 0;

    public ClickButton() {
        super("0");

        this.addActionListener(e -> {
            ++clicks;
            this.setText(clicks + "");
        });
    }
}
```

Dann benutzen wir unsere neue Buttonklasse im Fenster `ClickFrame`:

```
ClickButton clickButton = new ClickButton();
```

Damit verwalten wir die Informationen und ihre Darstellung (also das zur Information gehörende Verhalten) am selben Ort. Das macht das Programm besser wartbar und erweiterbar.

Übung: Implementieren und testen sie diese Variante.

Variante 3: Unterklasse von JButton als Abhörer

Anstatt einen Lambda-Ausdruck zu verwenden, um den Abhörer zu definieren, können wir auch ein bereits existierendes Objekt als Abhörer verwenden. Dazu muss dieses Objekt nur zusätzlich die Schnittstelle `ActionListener` implementieren. Hierfür bietet sich der Button selbst an, denn er hat Zugriff auf die notwendigen Informationen (Abbildung 15).

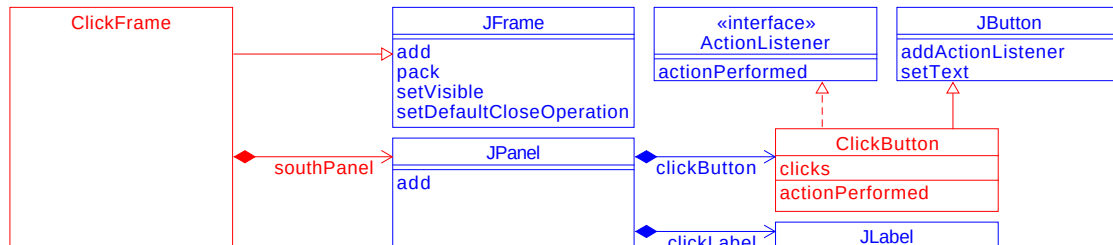


Abbildung 15: UML-Entwurfsmodell für ein Fenster `ClickFrame` mit einer Unterklasse von `JButton` als Ereignisabhörer.

Um die Schnittstelle `ActionListener` zu implementieren, müssen wir die Methode `actionPerformed` implementieren. In ihr steht der Code, der vorher im Lambda-Ausdruck stand, und der bei jeder Interaktion mit dem Button ausgeführt wird.

```
public class ClickButton extends JButton implements ActionListener {

    private int clicks = 0;

    public ClickButton() {
        super("0");

        this.addActionListener(this);
    }

    public void actionPerformed(ActionEvent e) {
        ++clicks;
        this.setText(clicks + "");
    }
}
```

An `addActionListener` übergeben wir mit `this` den Button als Ereignisabhörer von sich selbst (denn der Button ist ja auch die Ereignisquelle).

Übung: Implementieren und testen sie diese Variante.

Variante 4: Fenster als Abhörer

Eine weitere Variante wäre, das Fenster als Ereignisabhörer des Buttons zu verwenden. Dazu würde dann das Fenster die Schnittstelle `ActionListener` implementieren. In diesem Fall wäre

es wiederum besser, das Attribut `clicks` im Fenster zu verwalten (überlegen Sie!). Diese Lösung macht immer dann Sinn, wenn das Fenster auch noch andere Buttons als Ereignisquellen abhört, denn dann befindet sich die Ereignisbehandlung für alle Button-Interaktionen übersichtlich in der `actionPerformed`-Methode an einem Ort (Abbildung 16).

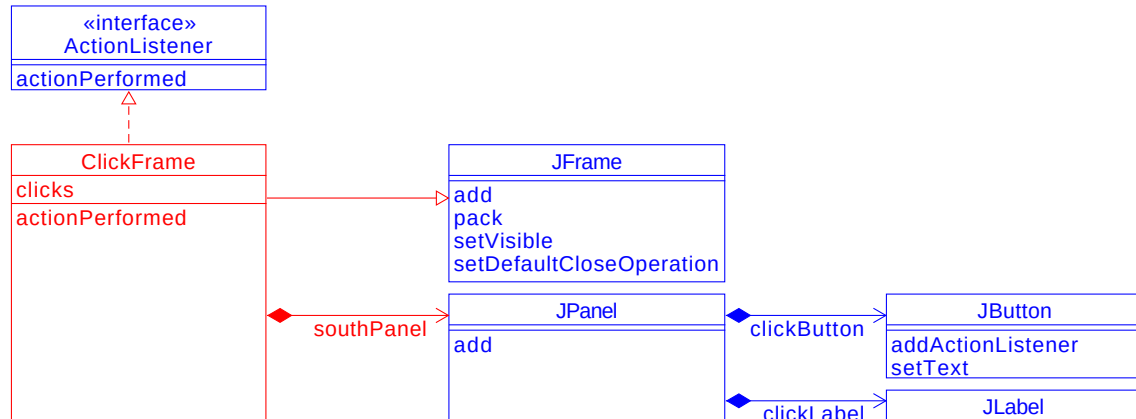


Abbildung 16: UML-Entwurfsmodell für ein Fenster ClickFrame, das selbst der Ereignisabhörer ist.

Da die Schnittstelle `ActionListener` also nun vom Fenster implementiert wird, muss dieses deshalb die Methode `actionPerformed` implementieren. Um in dieser auf den Button `clickButton` zugreifen zu können, legen wir diesen in dieser Variante als Attribut des Fensters an.

```

public class ClickFrame extends JFrame implements ActionListener {

    private int clicks = 0;
    private JButton clickButton;

    public ClickFrame() {
        super("ClickFrame");
        JPanel southPanel = new JPanel();
        add(southPanel, BorderLayout.SOUTH);
        JLabel clickLabel = new JLabel("Click me: ");
        southPanel.add(clickLabel);
        clickButton = new JButton(clicks + "");
        southPanel.add(clickButton);

        clickButton.addActionListener(this);

        setDefaultCloseOperation(DISPOSE_ON_CLOSE);
        pack();
        setVisible(true);
    }

    public void actionPerformed (ActionEvent e) {
        ++clicks;
    }
}

```

```

        clickButton.setText(clicks + " ");
    }
}

```

Nehmen wir an, ein Fenster hätte mehrere Buttons und soll als Abhörer für alle dieses Buttons fungieren. Wie kann in diesem Fall das Fenster unterscheiden, um welches Ereignis es sich jeweils handelt, also welcher der Buttons gedrückt wurde. Dazu sehen wir uns die `actionPerformed`-Methode einmal genauer an: Diese hat nämlich ein `ActionEvent`-Objekt `e` als Eingabeparameter. In diesem Ereignis-Objekt sind alle Informationen über das Ereignis gespeichert. Zum Beispiel können wir mit `e.getSource()` die Ereignisquelle und mit `e.getActionCommand()` die Beschriftung des Buttons abfragen.

Übung: Implementieren und testen sie diese Variante. Legen Sie dann einen zweiten Button an und hören beide Buttons durch das Fenster ab.

Variante 5: Eigene Abhörer-Klasse

Schließlich kann man auch eine zusätzliche spezielle Abhörerklasse `ClickButtonListener` zum Abhören der Benutzerinteraktion definieren. Abbildung 17 zeigt das Vorgehen als UML-Klassenkarte. Hier sieht man eine Objektbeziehung zwischen `ClickButtonListener` und `ClickButton` (Pfeil zwischen diesen beiden Klassen). Dieser bedeutet, dass jedes `ClickButtonListener`-Objekt ein `ClickButton`-Objekt kennt. Das `ClickButton`-Objekt hat in dieser Objektbeziehung die Rolle `b`. Über diese Objektbeziehung kann das Abhörer-Objekt auf das Button-Objekt zugreifen und dessen Beschriftung ändern. Aber wie greifen wir auf den Wert des Attributs `clicks` des Button-Objekts zu und erhöhen diesen? Da dieses Attribut privat ist, können wir nämlich nicht direkt darauf zugreifen! Deshalb müssen wir die Klasse `ClickButton` um eine Methode `getClicks` zur Abfrage des Werts und eine Methode `increment` zur Erhöhung des Werts ergänzen. Diese Methoden können wir dann über `b` im Abhörer aufrufen.

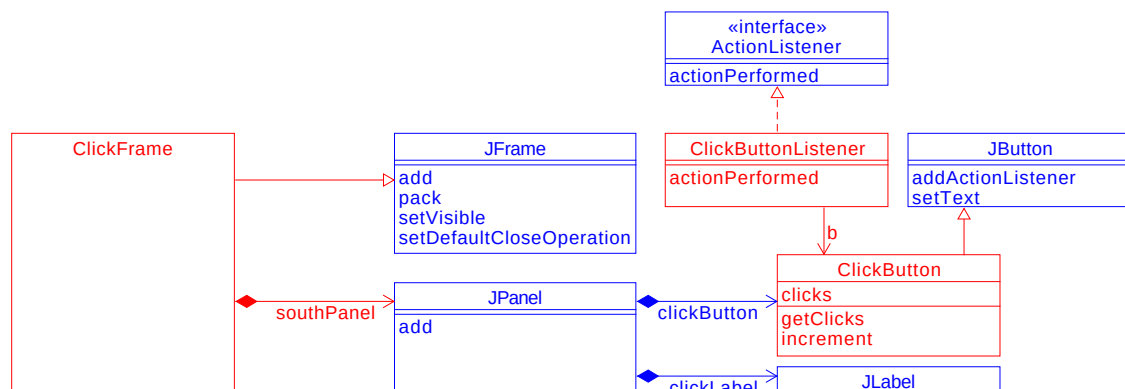


Abbildung 17: UML-Entwurfsmodell für ein Fenster `ClickFrame` mit einer separaten Abhörerklasse.

In unserer Implementierung legen dafür wir ein Attribut `b` an und übergeben `b` an den Abhörer. Die neue Abhörerklasse wird damit wie folgt implementiert:

```
public class ClickButtonListener implements ActionListener {

    private ClickButton b;

    public ClickButtonListener(ClickButton b) {
        this.b = b;
    }

    public void actionPerformed(ActionEvent e) {
        b.increment();
        b.setText(b.getClicks() + " ");
    }

}
```

Anschließend registrieren wir ein Objekt dieser Abhörerklasse bei der Ereignisquelle im Konstruktor der Klasse `ClickButton`:

```
this.addActionListener(new ClickButtonListener(this));
```

Diese letzte Variante scheint am aufwändigsten zu sein. Überlegen Sie, in welchen Fällen diese Variante trotzdem Vorteile bietet.

Übung: Sie sollten spätestens jetzt zur Übung alle vorgestellten Varianten einmal komplett durchprogrammieren.

8.3.2 Behandlung von Benutzerinteraktionen

Nach dieser kurzen Vorstellung und Diskussion einiger Beispiele wollen wir uns wieder systematisch einen kompletten Überblick verschaffen über die Behandlung von Benutzerinteraktionen. Um auf Benutzeraktionen reagieren zu können, nimmt man eine sogenannte **Ereignisbehandlung** vor.

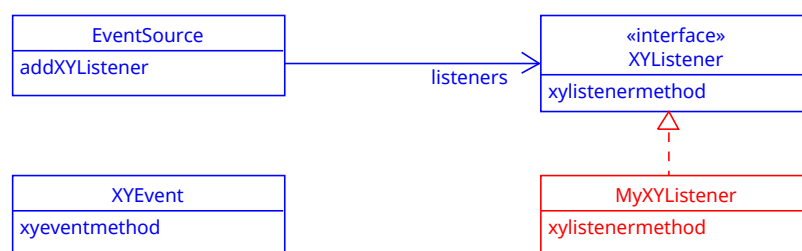


Abbildung 18: UML-Schema für die Java-Ereignisbehandlung.

Abbildung 18 zeigt einen Überblick, wie wir dazu die API benutzen (dabei sind API-Klassen blau und eigene Klassen rot dargestellt):

1. Jede grafische Komponente, mit der der Benutzer interagieren kann, ist eine Ereignisquelle (EventSource). Jede Benutzerinteraktion mit einer Ereignisquelle löst ein Ereignis bestimmter Art (XYEvent) aus.
Beispiel: Ein Button ist eine Ereignisquelle. Der Klick auf einen Button löst ein Actionevent aus.
2. Ein Ereignis ist ein Java-Objekt, das Informationen über die Benutzerinteraktion verwaltet, zum Beispiel über die Ereignisquelle. Solche Informationen können über Methoden der Ereignisklasse (xyeventmethod) abgefragt werden.
Beispiel: Für ein Actionevent durch einen Button-Klick kann mit getSource abgefragt werden, welcher Button geklickt wurde, und mit getActionCommand, welche Beschriftung der Button hat.
3. Jede Ereignisquelle verwaltet für jede Ereignisart eine Liste mit angemeldeten Ereignisabhörern (listeners). Diese ist standardmäßig zu Beginn leer.
Beispiel: Ein Button verwaltet eine Liste mit angemeldeten ActionListenern.
4. Für jede Ereignisart gibt es eine Schnittstelle für Ereignisabnehmer (XYListener). Diese gibt abstrakte Methoden zur Ereignisbehandlung (xylistenermethod) vor.
Beispiel: Für Actionevents gibt ActionListener als Ereignisabnehmer-Schnittstelle. Diese Schnittstelle gibt die abstrakte Methode actionPerformed vor.
5. Interagiert ein Benutzer mit einer Ereignisquelle, führt diese für alle bei ihr angemeldeten Ereignisabnehmer eine zugeordnete Methode zur Ereignisbehandlung aus.
Beispiel: Bei Klick auf einen Button führt dieser für alle bei ihr angemeldeten ActionListener die actionPerformed-Methode aus.
6. Damit das klappt, legen wir einen eigenen Ereignisabnehmer an (MyXYListener) und implementieren für diesen passend die zugeordnete Methode zur Ereignisbehandlung. Die Methode enthält also den Code, der im Falle eines Ereignisses ausgeführt werden soll. Dabei können wir Methoden des erstellten Ereignis-Objekts benutzen.
Beispiel: Um auf einen Button-Klick zu reagieren, erstellen wir einen ActionListener und implementieren dazu die Methode actionPerformed. Bei der Implementierung können wir getSource und getActionCommand des ausgelösten Actionevents benutzen.
7. Zudem melden wir vorher unseren Ereignisabnehmer mit addXYListener bei der Ereignisquelle an.
Beispiel: Bei einem Button melden wir mit der addActionListener-Methode unseren ActionListener an.

8.3.3 Ereignisarten

Zu jeder Ereignisart gibt es in Java eine Ereignisklasse. Abbildung 19 zeigt eine Übersicht über die Vererbungshierarchie der Ereignisklassen. Die gelb markierten Ereignisklassen untersuchen wir in dieser Vorlesung genauer.

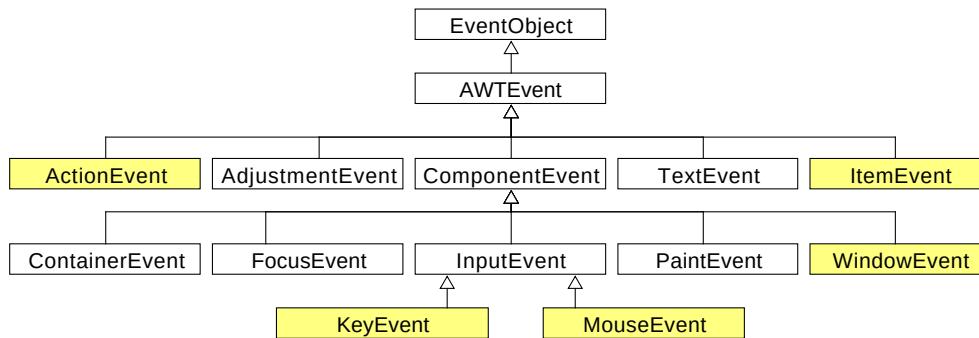


Abbildung 19: Vererbungshierarchie einiger Ereignisklassen.

Die gemeinsame Oberklasse aller Ereignisklassen ist `EventObject`. Diese stellt als gemeinsame Funktionalität den Zugriff auf die Ereignisquelle mit `getSource` zur Verfügung.

Das Abhören der verschiedenen Ereignisarten funktioniert nach demselben Konzept wie bei `ActionEvents`. Allerdings kann es sein, dass ein Abhörer für eine bestimmte Ereignisart nicht nur eine, sondern mehrere Methoden zur Ereignisbehandlung zur Verfügung stellt (und deshalb keine funktionale Schnittstelle mehr ist). Dann hängt es von der Art der Benutzeraktion ab, welche der Abhörermethoden ausgeführt wird.

Button-/Menü-Ereignisse: `ActionEvent`

Die Klasse `ActionEvent` haben wir schon im Beispiel kennengelernt und diskutiert. Wir fassen unser Wissen noch einmal in einem Steckbrief zusammen:

Definition 8.6: Steckbrief `ActionEvent`

Ausgelöstes Ereignis	<code>ActionEvent</code>
Ereignisabhörer-Schnittstelle	<code>ActionListener</code>
Abhörer anmelden	<code>addActionListener</code>
Abhörer-Methode(n)	<code>actionPerformed</code>
Ereignis abfragen	<code>getActionCommand</code>
Ereignisquelle	<code>JButton</code> <code>JMenuItem</code>

Neben Schaltflächen sind Menüeinträge mögliche Ereignisquellen eines `ActionEvent`. Mit der `getActionCommand`-Methode lässt sich die Beschriftung der grafischen Komponente (Schaltfläche oder Menüeintrag), mit der der Benutzer interagiert hat, abfragen. Diese Beschriftung beschreibt in der Regel die Aktion (oder den Befehl), die ausgeführt werden soll.

Haken-/DropDown-Ereignisse: ItemEvent

Ein ItemEvent-Objekt wird erzeugt bei (De-)Aktivierung verschiedener grafischer Komponenten, wie zum Beispiel einer Checkbox. Mit der getStateChange-Methode lässt sich abfragen, ob eine Aktivierung oder Deaktivierung stattgefunden hat. Die zugehörige Abhörer-Schnittstelle ist wie bei ActionEvents funktional.

Definition 8.7: Steckbrief ItemEvent

Ausgelöstes Ereignis	ItemEvent
Ereignisabhörer-Schnittstelle	ItemListener
Abhörer anmelden	addItemListener
Abhörer-Methode(n)	itemStateChanged
Ereignis abfragen	getStateChange
Ereignisquelle	JCheckbox JComboBox

Übung: Probieren Sie die Reaktion auf ItemEvents nun selbst aus, zum Beispiel, indem Sie ein Programm mit einer Checkbox erstellen, deren Hintergrundfarbe wechselt, wenn sie aktiviert / deaktiviert wird.

Tastatur-Ereignisse: KeyEvent

Ein KeyEvent-Objekt wird erzeugt beim Tippen auf eine Taste der Tastatur. Die Ereignisquelle ist dabei immer die aktuell aktivierte grafische Komponente. Mit den getKeyChar- und getKeyCode-Methoden lässt sich abfragen, auf welche Taste getippt wurde.

Definition 8.8: Steckbrief KeyEvent

Ausgelöstes Ereignis	KeyEvent
Ereignisabhörer-Schnittstelle	KeyListener
Abhörer anmelden	addKeyListener
Abhörer-Methode(n)	keyPressed keyTyped keyReleased
Ereignis abfragen	getKeyChar getKeyCode
Ereignisquelle	Component

Die zugehörige Abhörer-Schnittstelle ist hier nicht funktional, sondern stellt drei verschiedene Methoden bereit, die abhängig von der Art der Benutzeraktion aufgerufen werden:

1. keyTyped-Methode:

Wird bei Eingabe eines Unicode-Zeichens (durch Betätigen einer Taste oder einer Tastenkombination; plattform- und tastaturunabhängig) aufgerufen. Diese Methode reagiert aber nicht bei Betätigen von Aktions- oder Modifikationstasten wie zum Beispiel F5, Alt oder

Enter. Das eingegebene Unicode-Zeichen kann mit der KeyEvent-Methode `getKeyChar` abgefragt werden.

2. `keyPressed`-Methode:

Wird bei Drücken einer beliebigen Taste aufgerufen (plattform- und tastaturabhängig). Die gedrückte Taste kann mit der KeyEvent-Methode `getKeyCode` abgefragt werden. Dabei werden auch Aktions- und Modifikationstasten erkannt.

3. `keyReleased`-Methode:

Ähnlich wie `keyPressed`, wird aber beim Loslassen einer beliebigen Taste aufgerufen (plattform- und tastaturabhängig). Die Taste kann ebenso mit der KeyEvent-Methode `getKeyCode` abgefragt werden.

Übung: Probieren Sie die Reaktion auf Keyevents nun selbst aus. Erstellen Sie zum Beispiel ein Programm mit einem Label, das immer die aktuell gedrückte Taste anzeigt. Dabei müssen Sie übrigens alle Methoden `keyTyped`, `keyPressed` und `keyReleased` implementieren, auch wenn Sie nicht alle davon benutzen (warum?). Nicht benötigte Methoden erhalten dabei eine leere Implementierung. Dazu lernen wir gleich mit Adapterklassen noch eine Möglichkeit kennen, ohne solche leeren Implementierungen auszukommen.

Fenster-Ereignisse: `WindowEvent`

Ein `WindowEvent`-Objekt wird erzeugt, wenn sich durch Interaktion mit einem Fenster dessen Zustand ändert, zum Beispiel beim Klicken auf Symbole zum Schließen, Minimieren und Maximieren. Mit den `getNewState`- und `getOldState`-Methoden lassen sich der alte und der neue Zustand abfragen. Die Abhörer-Schnittstelle `WindowListener` definiert sieben Methoden zur Behandlung von Fenstereignissen:

1. `windowActivated`-Methode:

Wird aufgerufen, wenn ein Fenster aktiviert wird.

2. `windowDeactivated`-Methode:

Wird aufgerufen, wenn ein Fenster deaktiviert wird.

3. `windowClosed`-Methode:

Wird aufgerufen, wenn ein Fenster mit `dispose` geschlossen wird.

4. `windowClosing`-Methode:

Wird aufgerufen, wenn ein Fenster über eine Mausinteraktion geschlossen wird. Die Implementierung können wir für die Klassen `JFrame` und `JDialog` umgehen, indem wir wie im Beispiel die Methode `setDefaultCloseOperation` benutzen.

5. `windowOpened`-Methode:

Wird aufgerufen, wenn ein Fenster das erste Mal mit `setVisible(true)` sichtbar gemacht wird.

6. `windowIconified`-Methode:

Wird aufgerufen, wenn ein Fenster minimiert wird.

7. `windowDeiconified`-Methode:

Wird aufgerufen, wenn ein Fenster vom minimierten Zustand wieder vergrößert wird.

Definition 8.9: Steckbrief `WindowEvent`

Ausgelöstes Ereignis	<code>WindowEvent</code>
Ereignisabhörer-Schnittstelle	<code>WindowListener</code>
Abhörer anmelden	<code>addWindowListener</code>
Abhörer-Methode(n)	<code>windowActivated</code> <code>windowDeactivated</code> <code>windowClosed</code> <code>windowClosing</code> <code>windowOpened</code> <code>windowIconified</code> <code>windowDeiconified</code>
Ereignis abfragen	<code>getNewState</code> <code>getOldState</code>
Ereignisquelle	<code>Window</code>

Wie bei Keyevents haben also auch Abhörer von Windowevents mehrere Methoden, deren Aufruf von der Art der Benutzerinteraktion mit dem Fenster abhängt, und von denen man in der Regel nicht alle benötigt. Um sich die leere Implementierung aller nicht benötigten Methoden zu sparen, gibt es zu jeder Abhörer-Schnittstelle mit mehreren Methoden eine zugehörige **Adapterklasse**, die die Methoden der Abhörer-Schnittstelle **leer implementiert**. Benötigt man also nicht alle Methoden der Abhörer-Schnittstelle, so implementiert man eine Unterklasse der Adapterklasse und überschreibt nur die benötigten Methoden.

Zum Beispiel gibt es eine Adapterklasse `WindowAdapter` mit leeren Implementierungen aller sieben `WindowListener`-Methoden. Eine Abhörerklasse `MyWindowListener` lässt sich dann wie folgt erstellen:

```
public class MyWindowListener extends WindowAdapter {  
    // Benötigte Methoden überschreiben  
}
```

Beachten Sie, dass Adapterklassen **keine** Schnittstellen sind und somit **nicht** über das Schlüsselwort `implements` eingebunden werden. Sie können ausschließlich über das Schlüsselwort `extends` in eine neue Klasse übernommen werden.

Alternativ kann man eine Abhörerklasse auch als eine sogenannte **anonyme innere** Klasse des Fensters implementieren:

```
public class MyFrame extends JFrame {  
    public MyFrame() {  
  
        // Layout  
  
        this.addWindowListener(new WindowAdapter() {  
            // Benötigte Methoden überschreiben  
        })  
    }  
}
```

```

    });
}
}

```

Mit der Anweisung

```

new WindowAdapter(){
    // Benötigte Methoden überschreiben
}

```

wird ein Objekt einer anonymen Unterklasse der Klasse `WindowAdapter` erzeugt. Im Rumpf überschreibt man die gewünschten Abhörer-Methoden. Das erzeugte Objekt wird dann an

```

addWindowListener(...)

```

übergeben und somit als Ereignisabhörer angemeldet. Da die Unterklasse keinen Namen hat, spricht man von einer **anonymen** Klasse. Außerdem ist die Abhörerklasse eine sogenannte **innere Klasse**, da sich ihre Implementierung innerhalb der Fensterklasse befindet. Damit ist der Abhörer nur in diesem Fenster, aber nicht für andere Fenster und Anwendungen nutzbar. Man muss für den Einzelfall abwägen, ob der Einsatz einer anonymen inneren Klasse sinnvoll ist oder nicht. Die genaue Definition innerer Klassen und deren verschiedener Varianten und Einsatzgebiete ist nicht Stoff dieser Vorlesung.

Definition 8.10: Adapterklassen

Eine **Adapterklasse** bietet eine leere Implementierung einer Schnittstelle an.

Statt die Schnittstelle zu implementieren, kann man eine Unterklasse der Adapterklasse bilden und die benötigten Schnittstellen-Methoden überschreiben.

Das ist sinnvoll, wenn man nicht alle Methoden der Schnittstelle benötigt.

Übung: Experimentieren Sie nun selbstständig mit den verschiedenen präsentierten Varianten, ein Fenster schließbar zu machen:

1. Mit der `setDefaultCloseOperation`-Methode des Fensters.
2. Mit einem Abhörer für Fensterereignisse, wobei
 - a) das Fenster sich selbst abhört.
 - b) der Abhörer durch eine separate Klasse, die die Schnittstelle `WindowListener` implementiert, definiert wird.
 - c) der Abhörer durch eine allgemein nutzbare Unterklasse von `WindowAdapter` implementiert wird.
 - d) der Abhörer durch eine anonyme innere Unterklasse von `WindowAdapter` implementiert wird.

Maus-Ereignisse: MouseEvent

Ein MouseEvent-Objekt wird erzeugt, wenn der Benutzer mit der Maus in eine grafische Komponente klickt, oder die Maus über einer grafischen Komponente bewegt. Bei Mausbewegungen werden solche Mouseevents dabei in sehr kurzen Zeitabständen (im Bereich von Nanosekunden) entlang der Mausbewegung erzeugt. Mit den getX- und getY-Methoden lassen sich die Koordinaten der zum Mausereignis gehörigen Mausposition abrufen. Diese Position wird relativ zur linken oberen Ecke der zugehörigen Komponente berechnet.

Die Abhörer-Schnittstelle `MouseListener` definiert fünf Methoden zur Behandlung von Mauser-eignissen **ohne Mausbewegung**:

1. `mouseClicked`-Methode:
Wird aufgerufen, wenn auf eine Maustaste in einer grafischen Komponente **geklickt** (direkt nacheinander gedrückt und losgelassen) wird.
2. `mousePressed`-Methode:
Wird aufgerufen, wenn eine Maustaste in einer grafischen Komponente **gedrückt** wird.
3. `mouseReleased`-Methode:
Wird aufgerufen, wenn die Maustaste in einer grafischen Komponente **losgelassen** wird.
4. `mouseEntered`-Methode:
Wird aufgerufen, wenn sich die Maus in eine grafische Komponente **hineinbewegt**.
5. `mouseExited`-Methode:
Wird aufgerufen, wenn sich die Maus aus einer grafischen Komponente **herausbewegt**.

Da meist nur ein oder zwei dieser Methoden benötigt werden, stellt die API wieder eine Adapterklasse `MouseAdapter` mit leeren Implementierungen aller `MouseListener`-Methoden zur Verfügung.

Definition 8.11: Steckbrief MouseEvent

Ausgelöstes Ereignis	MouseEvent
Ereignisabhörer-Schnittstelle	MouseListener
Abhörer anmelden	addListener
Abhörer-Methode(n)	mousePressed MouseClicked mouseReleased mouseExited mouseEntered
Ereignisabhörer-Schnittstelle	MouseMotionListener
Abhörer anmelden	addMouseMotionListener
Abhörer-Methode(n)	mouseMoved mouseDragged
Ereignis abfragen	getX getY
Ereignisquelle	Component

Mausereignisse **mit Mausbewegung** werden von der Schnittstelle MouseMotionListener behandelt, die die folgenden zwei Methoden definiert:

1. mouseDragged-Methode:
Wird durchgehend jeweils mit Abstand weniger Sekundenbruchteile aufgerufen, solange die Maus **mit** gedrückter Maustaste bewegt wird.
2. mouseMoved-Methode:
Wird durchgehend jeweils mit Abstand weniger Sekundenbruchteile aufgerufen, solange die Maus **ohne** gedrückte Maustaste bewegt wird.

Wahrscheinlich vermissen Sie jetzt einige übliche Funktionalitäten wie das Öffnen eines Popup-Menüs bei einem Rechtsmausklick. Bisher haben wir nämlich nicht zwischen der linken und der rechten Maustaste unterschieden. Auch das können wir aber über das Mausereignis abfragen. Ein Rechtsklick wird auf unterschiedlichen Systemen auf verschiedene Arten erkannt, und zwar in mousePressed oder in mouseReleased, so dass eine Überprüfung grundsätzlich in beiden Methoden vorgenommen werden sollte. In modernen Systemen gibt bei einem Rechtsklick die MouseEvent-Methode isPopupTrigger den Wert true zurück.

Daneben können wir auch direkt abfragen, welche Maustaste gedrückt wurde: Mit

```
SwingUtilities.isRightMouseButton(e)
```

fragen wir für eine MouseEvent `e` ab, ob die rechte Maustaste gedrückt wurde. Mit

```
SwingUtilities.isMiddleMouseButton(e)
```

fragen wir für eine MouseEvent `e` ab, ob die mittlere Maustaste gedrückt wurde.

Im Weiteren benötigen wir Mausereignisse vor allem zum Zeichnen, welches wir im Unterkapitel 8.5 besprechen.

8.4 Fenster und Dialoge zu Datenklassen

Wir wollen nun unser neu gewonnenes Wissen anwenden, um für die begonnene Angestelltenverwaltung einen Teil der grafischen Benutzerschnittstelle anzulegen. Anschließend werden wir einige allgemeine Regeln zur Erstellung von Fenstern zu Datenklassen ableiten und kurz die Unterstützung von Standarddialogen durch die API diskutieren.

8.4.1 Fallstudie: Angestelltenverwaltung

Wir beginnen mit einer Übersicht in UML über drei Fenster der grafischen Benutzerschnittstelle mit ihrer Anbindung an die schon entwickelten Datenklassen in Abbildung 20:

1. Dem Hauptanwendungsfenster EmployeeGUI.
2. Einem Dialog zum Anzeigen aller Angestellter ListEmployee. In diesem Dialog soll der Benutzer zudem die Möglichkeit haben, einen ausgewählten Angestellten zu löschen.
3. Einem Dialog RegisterEmployee zum Erfassen eines neuen Angestellten.

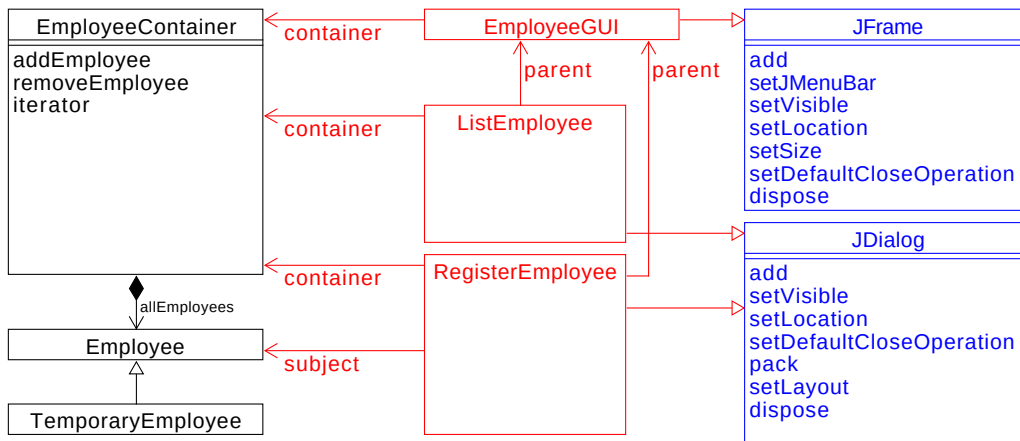


Abbildung 20: Fensterklassen und deren Anbindung an die Datenklassen der Angestelltenverwaltung.

Das Hauptanwendungsfenster EmployeeGUI ist eine Unterklasse von JFrame. Damit startet die Anwendung. Alle weiteren Fenster sind untergeordnete Dialoge mit EmployeeGUI als Elternfenster (Rolle parent) und JDialog als Oberklasse. Die Klassen JFrame und JDialog stellen viele nützliche Standardmethoden bereit, die wir in unseren neuen Unterklassen verwenden können (und von denen wir einige im Diagramm angegeben haben).

Alle drei Fenster haben über die Rolle container Zugriff auf den Angestelltencontainer:

- Im Hauptanwendungsfenster benötigen wir den Zugriff, um den Container an die beiden Dialoge weiterzugeben. In dieser Version des Programms werden wir hier den Container neu erstellen und immer mit einem leeren Container beginnen. In späteren Versionen werden wir die Möglichkeit hinzunehmen, den Container aus einer Textdatei zu befüllen.
- Im Listenfenster benötigen wir den Zugriff, um die Daten aus dem Container anzeigen zu können. Dazu werden wir das Iteratormuster verwenden. Zudem soll es möglich sein, einen ausgewählten Angestellten mit `removeEmployee` aus dem Container zu löschen.
- Im Erfassungsfenster benötigen wir den Zugriff, um einen neu angelegten Angestellten mit `addEmployee` in den Container aufzunehmen.

Zudem hat das Erfassungsfenster über die Rolle `subject` Zugriff auf einen Angestellten. Das ist der Angestellte, den wir dort neu anlegen wollen.

Man sieht dass die Fensterklassen nur in eine Richtung an die Datenklassen angebunden sind:

Die Fensterklassen haben Zugriff auf die Datenklassen, aber die Datenklassen haben keinen Zugriff auf die Fensterklassen.

Das hat einige Vorteile: Bei der Anbindung der grafischen Benutzerstelle müssen wir nichts mehr an den Datenklassen ändern. Ändern wir zudem später einmal die grafische Benutzerstelle, müssen wir auch nichts an den Datenklassen ändern.

Hauptanwendungsfenster

In Abbildung 21 sehen wir uns nun ein UML-Diagramm mit einer genaueren Darstellung des Hauptanwendungsfensters inklusive Anbindung an API-Klassen an:

- Aus dem Diagramm geht nicht das Layout hervor. Aber man sieht mit dem Label `message`, der Schaltfläche `close` und der Menübar `mb` alle im Fenster enthaltenen grafischen Komponenten, inklusive dem hierarchischen Aufbau der Menübar mit dem Menü `new`, das die zwei Menüeinträge `showall` und `register` enthält.
- Man sieht weiter, dass das Fenster als Ereignisabhörer für `ActionEvents` fungiert. Die Ereignisquellen hierfür sind die Schaltfläche und die beiden Menüeinträge. Für jede diese Ereignisquellen sehen wir eine separate **On-Methode** vor, die wir ausführen wollen, wenn die Ereignisquelle ein Ereignis auslöst. Zum Beispiel wollen wir `onClose` aufrufen, wenn der Benutzer auf die Schaltfläche `close` klickt.

Das Hauptanwendungsfenster hat ein Referenzattribut `container` zur Anbindung der Datenklassen. Dieser wird in dieser Version der Anwendung bei Start der Anwendung jedes Mal leer initialisiert. Später werden wir die Anwendung um die Möglichkeit erweitern, bei Start der Anwendung abgespeicherte Angestellte in den Container zu laden.

```
public class EmployeeGUI extends JFrame implements ActionListener {
    private EmployeeContainer container = new EmployeeContainer();
```

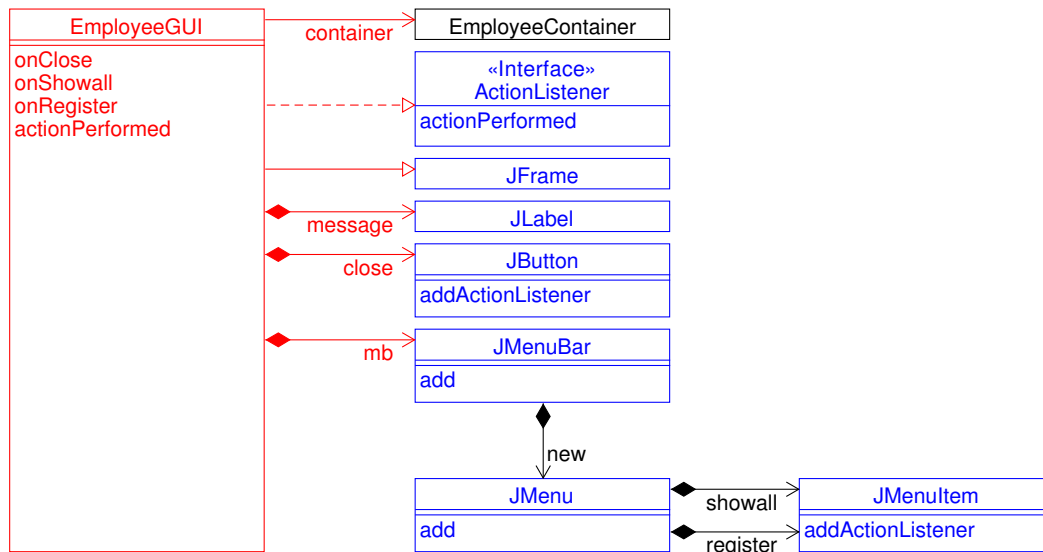


Abbildung 21: Das Hauptanwendungsfenster der Angestelltenverwaltung.

```

public EmployeeGUI() { ... }

public void actionPerformed(ActionEvent e) { ... }

private void onRegister() { ... }

private void onShowall() { ... }

private void onClose() { ... }
}

```

Exemplarisch zeigen wir die Implementierung der `actionPerformed`-Methode. Hier nutzen wir die `getActionCommand`-Methode der Klasse `ActionEvent`, um zwischen den verschiedenen Ereignisquellen zu unterscheiden, und rufen für jede Ereignisquelle die zugehörige On-Methode auf.

```

public void actionPerformed(ActionEvent e) {
    if (e.getActionCommand().equals("Show All Employees")) {
        onShowall();
    } else if (e.getActionCommand().equals("Register New Employee")) {
        onRegister();
    } else if (e.getActionCommand().equals("Close Application")) {
        onClose();
    }
}

```

In der `onClose`-Methode schließen wir das Fenster und die Anwendung:

```

private void onClose() {

```

```

        dispose();
        System.exit(0);
    }

```

In der `onShowall`-Methode öffnen wir den Listendialog, wobei wir das Hauptanwendungsfenster als Elternfenster übergeben und den Container weiterreichen.

```

private void onShowall() {
    new ListEmployee(this, container);
}

```

In der `onRegister`-Methode öffnen wir den Erfassungsdialog.

Übung: Vervollständigen Sie nun selbst die Implementierung, indem Sie den Konstruktor des Fensters gemäß dem UML-Diagramm implementieren und die enthaltenen Komponenten nach einem Layout Ihrer Wahl anordnen. Sorgen Sie dabei dafür, dass sich das Fenster nicht über das Fensterkreuz schließen lässt (dafür ist der close-Button da).

Listendialog

In Abbildung 22 sehen wir uns ein UML-Diagramm mit einer genaueren Darstellung des Listenfensters inklusive Anbindung an API-Klassen an:

- Auch hier sehen wir alle im Fenster enthaltenen grafischen Komponenten. Das mit `Employee` parametrisierte `JList`-Objekt `list` ist zur Darstellung und Auswahl von Angestelltenobjekten da. Über zwei hierarchisch in einem Panel enthaltene Schaltflächen können Benutzeraktionen zum Löschen eines in `list` ausgewählten Angestellten (`delete`) und zum Abbrechen (`cancel`) ausgeführt werden.
- Das Fenster fungiert wieder als Ereignisabnehmer. Dafür ist wieder für jede Schaltfläche eine `On`-Methode vorgesehen.
- Das `JList`-Objekt wird nach Änderungen mit der `updateList`-Methode aktualisiert.

Auch der Listendialog hat ein Referenzattribut `container` zur Anbindung der Datenklassen. Dieser wird bei Start des Dialogs vom Hauptanwendungsfenster übergeben.

Zudem nehmen wir die grafische Komponente `list` zur Darstellung der Liste aller Angestellten als Referenzattribut auf. Das machen wir immer dann, wenn eine grafische Komponente in einer separaten Methode aktualisiert wird, um auf diese in dieser Methode Zugriff zu haben. Alle anderen grafischen Komponenten werden wie gewohnt nur im Konstruktor angelegt.

```

public class ListEmployee extends JDialog implements ActionListener {

    private EmployeeContainer container;
    private JList<Employee> list;

    public ListEmployee(EmployeeGUI parent, EmployeeContainer
        container) {

```

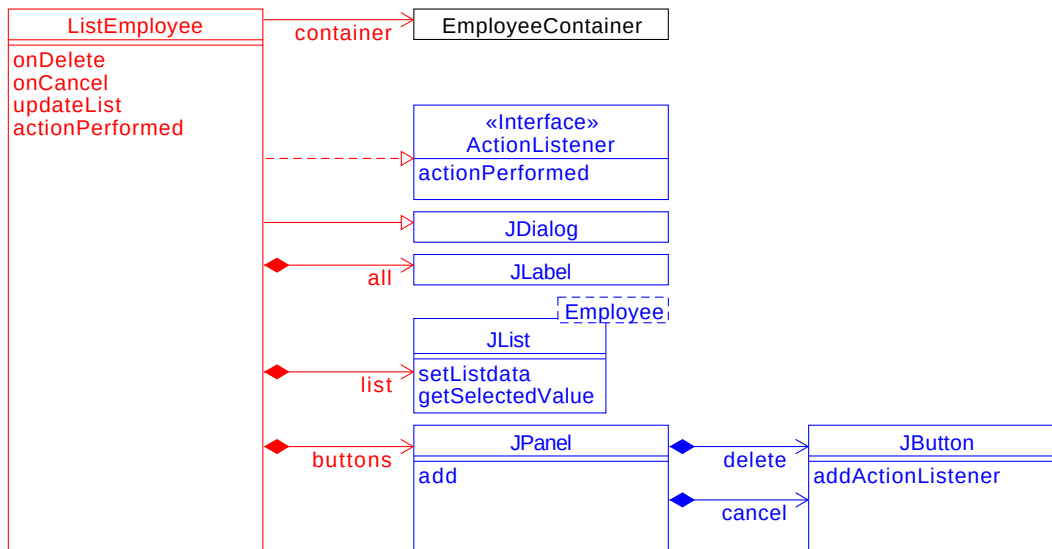


Abbildung 22: Das Listenfenster der Angestelltenverwaltung.

```

        super(parent, "Show All Employees", true);
        this.container = container;
        ...
    }

    private void updateList() { ... }

    public void actionPerformed(ActionEvent e) { ... }

    private void onDelete() { ... }

    private void onCancel() { ... }
}

```

Mit `super` rufen wir den Konstruktor der Oberklasse `JDialog` auf. Dieser erwartet:

- als ersten Parameter das Elternfenster: Dieses ist hier das Hauptanwendungsfenster.
- als zweiten Parameter den Fenstertitel ("Show All Employees").
- als dritten Parameter einen Wahrheitswert, der angibt, ob der Dialog modal ist. Wir erzeugen das Listenfenster hier als modalen Dialog (`true`). Bis zur Beendigung des Dialogs werden dadurch alle anderen Fenster der Anwendung blockiert.

Die Aktualisierung der Listendarstellung aller Angestellten sieht wie folgt aus. Diese wird einmal bei Initialisierung des Fensters und dann nach jeder Änderung an der Angestelltenliste aufgerufen. Letzteres ist der Fall, wenn ein Angestellter gelöscht wird.

```

private void updateList() {
    Vector<Employee> v = new Vector<Employee>();
    for (Employee e : container)

```

```

        v.add(e);
        list.setListData(v);
        pack();
    }

```

Ein `JList`-Objekt zeigt zeilenweise die String-Repräsentationen einer Liste von Objekten an. Dabei können einzelne Objekte mit der Maus aus der Liste ausgewählt werden. Das `JList`-Objekt wird mit dem Typ der dargestellten Objekte parametrisiert (hier: `Employee`). Die Liste der dargestellten Objekte kann als `Vector`-Objekt an die `setListData`-Methode des `JList`-Objekts übergeben werden. Wir erstellen und aktualisieren also die Liste, indem wir jeweils die aktuellen Angestellten aus dem Container in einem `Vector`-Objekt sammeln, und mit `setListData` dann den Inhalt des `JList`-Objekts neu mit diesem `Vector`-Objekt setzen. Danach rufen wir `pack` auf, da sich die Größe der grafischen Komponenten geändert haben kann und deshalb auch die Größe des Fenster angepasst werden muss.

Schließlich besprechen wir noch die Implementierung zum Löschen eines Angestellten:

```

private void onDelete() {
    Employee subject = list.getSelectedValue();
    if (subject != null) {
        container.removeEmployee(subject);
        updateList();
    } else {
        JOptionPane.showMessageDialog(this, "No employee selected", "
            Error", JOptionPane.ERROR_MESSAGE);
    }
}

```

Mit der `getSelectedValue`-Methode von `list` können wir abfragen, welchen Eintrag der Liste der Benutzer ausgewählt hat. Hat der Benutzer keine Auswahl getroffen, gibt die Methode `null` zurück.

Falls der Benutzer eine Auswahl getroffen hat, entfernen wir den ausgewählten Angestellten mit der `removeEmployee`-Methode aus dem Container und aktualisieren die Liste.

Falls der Benutzer keine Auswahl getroffen hat, wollen wir das mit einem Nachrichtendialog anzeigen. Solche Dialoge müssen wir nicht mühselig selbst programmieren. Vielmehr stellt die Klasse `JOptionPane` eine große Auswahl an Standarddialogen zur Verfügung, die über diverse statische **Show-Methoden** erstellt werden können. Hier erzeugen wir mit `showMessageDialog` wie folgt unseren Nachrichtendialog:

- als ersten Parameter übergeben wir die Elternkomponente des Nachrichtendialogs.
- als zweiten Parameter übergeben wir die Nachricht, die dargestellt werden soll.
- als dritten Parameter übergeben wir den Titel des Dialogs.
- als vierten Parameter übergeben wir die Art der Nachricht. Hierfür stehen die folgenden Klassenkonstanten zur Verfügung, die jeweils die Darstellung beeinflussen:
 - `ERROR_MESSAGE`: Für Fehlnachrichten.
 - `INFORMATION_MESSAGE`: Für Informationen.
 - `WARNING_MESSAGE`: Für Warnungen.

- `QUESTION_MESSAGE`: Für Fragen.
- `PLAIN_MESSAGE`: Für Dialoge ohne Icon.

Übung: Vervollständigen Sie selbstständig die restliche Implementierung der Klasse gemäß dem UML-Diagramm und ordnen Sie die enthaltenen Komponenten nach einem Layout Ihrer Wahl an. Sorgen Sie dabei wieder dafür, dass sich das Fenster nicht über das Fensterkreuz schließen lässt (dafür ist der cancel-Button da).

Erfassungsdialog

In Abbildung 23 sehen wir uns schließlich ein UML-Diagramm mit einer genaueren Darstellung des Erfassungsfensters inklusive Anbindung an API-Klassen an.

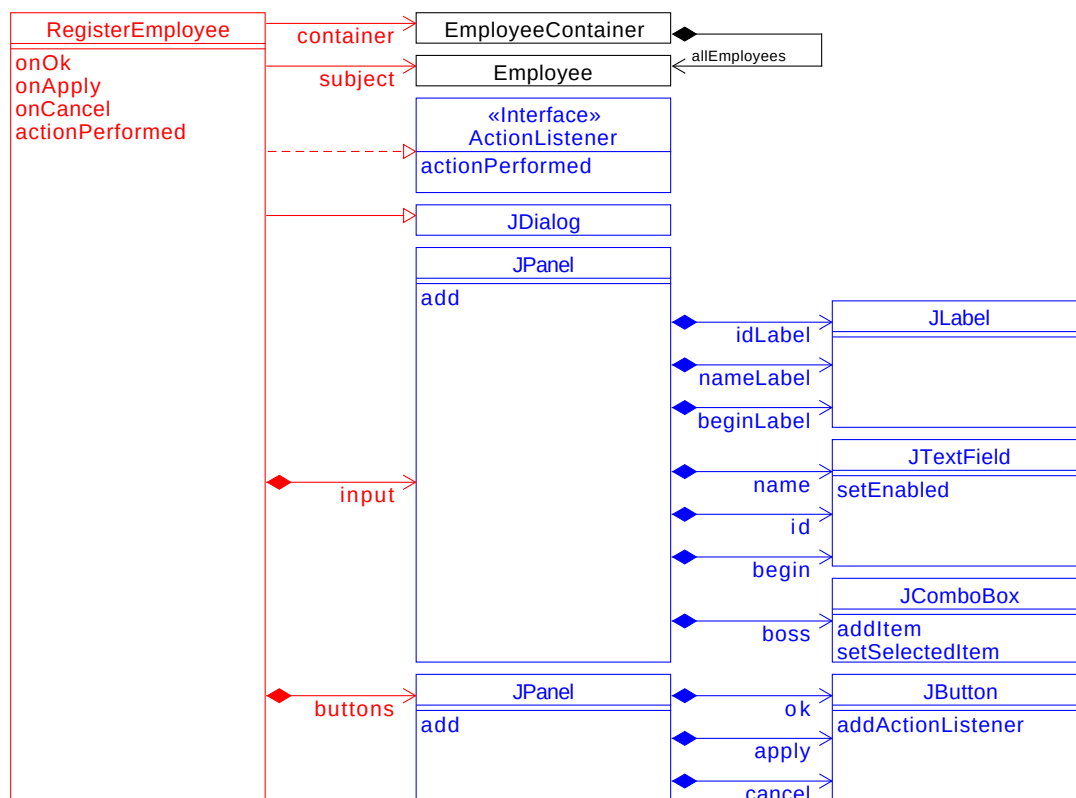


Abbildung 23: Das Erfassungsfenster der Angestelltenverwaltung.

Dem Diagramm kann man wieder die hierarchische Gliederung der enthaltenen Komponenten mit `JPanel`-Objekten entnehmen. Die Textfelder (`JTextField`) sind zur Eingabe und Anzeige der Daten eines neuen Angestellten da. Sie sind mit entsprechenden Beschriftungen (`JLabel`) beschriftet. Bei der Ereignisbehandlung können wir den Inhalt der Textfelder über die Methode `getText` abfragen. Der Vorgesetzte eines Angestellten kann über eine Auswahlliste (`JComboBox`) gesetzt werden.

Über die Schaltflächen (JButton) können gewünschte Benutzeraktionen ausgeführt werden. Diese werden wieder vom Fenster abgehört.

Der Erfassungsdialog hat neben einem Referenzattribut `container` zur Anbindung des Datencontainers noch ein Referenzattribut `subject` für den Zugriff auf den neu erfassten Angestellten.

Zudem nehmen wir alle Textfelder und die Auswahlliste für Vorgesetzte als Referenzattribute auf, da wir diese in verschiedenen Methoden aktualisieren.

Die Methoden `save` und `clear` sind zusätzliche private Hilfsmethoden, die später erklärt werden.

```
public class RegisterEmployee extends JDialog implements
    ActionListener {

    private Employee subject;
    private EmployeeContainer container;

    private JTextField id;
    private JTextField name;
    private JTextField begin;
    private JComboBox<Employee> boss;

    public RegisterEmployee(EmployeeGUI parent, EmployeeContainer
        container) {
        super(parent, "Register New Employee", true);
        this.container = container;
        ...

        id = new JTextField(5);
        id.setEnabled(false);
        id.setText(Employee.getIdCounter() + "");
        ...

        boss = new JComboBox<Employee>();
        boss.addItem(null);
        for (Employee e : container)
            boss.addItem(e);
        boss.setSelectedItem(null);
        ...
    }

    public void actionPerformed(ActionEvent e) { ... }

    private void onOk() { ... }

    private void onApply() { ... }

    private void save() { ... }

    private void onCancel() { ... }
```

```

        private void clear() { ... }
    }

```

Zur Implementierung des Konstruktors wollen wir uns zwei Aspekte genauer ansehen:

- Das Textfeld `id` für die ID ist nicht aktiviert (`id.setEnabled(false)`). Damit kann der Benutzer keine eigenen Einträge vornehmen. Stattdessen wird die ID automatisch auf den Zähler der Klasse `Employee` gesetzt (`Employee.getIdCounter()`).
- Der Auswahlliste `boss` fügen wir `null` hinzu (`boss.addItem(null)`). Dieses Element steht für *kein Vorgesetzter ausgewählt*. Standardmäßig ist dieses Element zu Beginn ausgewählt (`boss.setSelectedItem(null)`). Darüber hinaus fügen wir der Auswahlliste alle Angestellten aus dem Container hinzu. Der Benutzer kann diese Liste ausklappen und durch einen Mausklick ein Element auswählen. Diese Auswahl können wir in der Ereignisbehandlung über die Methode `getSelectedItem` abfragen.

Die beiden Methoden `onOk` und `onApply` sind beide dazu da, mit den eingegebenen Daten einen neuen Angestellten zu erstellen und in den Container aufzunehmen. Dies lagern wir in eine zusätzliche Methode `save` aus. Der Unterschied zwischen den beiden Methoden ist lediglich, dass nach dem Speichern in `onOk` der Dialog mit `onCancel` geschlossen wird, in `onApply` nicht.

```

private void onOk() {
    save();
    onCancel();
}

private void onApply() {
    save();
}

```

Schließlich betrachten wir das Erstellen eines Angestellten mit `save` genauer:

```

private void save() {
    try {
        DateTimeFormatter dtf = DateTimeFormatter.ofPattern("dd.MM.
            yyyy");
        LocalDate beginDate = LocalDate.parse(begin.getText(), dtf);
        subject = new Employee(name.getText(), beginDate);
        Employee newboss = (Employee) boss.getSelectedItem();
        if (newboss != null)
            subject.linkBoss(newboss);
        else
            subject.unlinkBoss();
        container.addEmployee(subject);
        boss.addItem(subject);
        pack();
        clear();
        subject = null;
    } catch (DateTimeParseException e) {
        JOptionPane.showMessageDialog(this, "Date not correct:\n" + e
            .getMessage(), "Error", JOptionPane.ERROR_MESSAGE);
    }
}

```

```

    } catch (IllegalArgumentException e) {
        JOptionPane.showMessageDialog(this, "Employee cannot be
        registered:\n" + e.getMessage(), "Error", JOptionPane.
        ERROR_MESSAGE);
    }
}

```

Alle Anweisungen befinden sich hier in einem `try`-Block, da es zu verschiedenen Ausnahmen kommen kann. Im Falle einer Ausnahme wird kein neuer Angestellter in den Container aufgenommen und stattdessen in einem der `catch`-Blöcke ein passender Warndialog angezeigt. Wir haben also alle Ausnahmen aus den Datenklassen bis an diese Stelle weitergereicht, um hier eine passende Fehlerbehandlung durchführen zu können.

Da das Datum für den Vertragsbeginn im Textfeld erstmal als String vorliegt, müssen wir daraus zuerst ein `LocalDate`-Objekt machen. Den Eintrag für den Namen können wir direkt aus dem Textfeld übernehmen und so einen neuen Angestellten erstellen. Dafür benutzen wir die Referenz `subject`. Danach setzen wir den Vorgesetzten gemäß der Auswahl in der Auswahlliste und nehmen den erstellten Angestellten in den Container auf. Zudem aktualisieren wir noch die Vorgesetzten-Auswahlliste für das Erfassen des nächsten Angestellten. Danach ist es erforderlich, das Fenster mit `pack` anzupassen, da sich die Auswahlliste verändert hat. Schließlich setzen wir mit `clear` die Inhalte aller Textfelder zurück und setzen die Referenz `subject` wieder auf `null`.

Übung: Vervollständigen Sie wieder selbstständig die restliche Implementierung gemäß dem UML-Diagramm und ordnen Sie die enthaltenen Komponenten nach einem Layout Ihrer Wahl an. Sorgen Sie auch wieder dafür, dass sich das Fenster nicht über das Fensterkreuz schließen lässt (dafür sind der `cancel`- und der `ok`-Button da).

Übung: Sie können die vorgestellten Konzepte zudem selbstständig weiter üben, indem Sie weitere Dialoge aus der Übersicht in Abbildung 3 implementieren, zum Beispiel zum Modifizieren eines im Listenfenster ausgewählten Angestellten, oder zum Anlegen eines Zeitangestellten.

8.4.2 Grundregeln und Ausblicke

Wir leiten aus der Fallstudie folgende allgemeinen Anmerkungen zur Erstellung von Fenstern zu Datenklassen ab.

Attribute von Fensterklassen

Wie wir in der Fallstudie gesehen haben, benötigen wir Attribute in von uns selbst implementierten Fenstern zu Datenklassen in drei Fällen:

- **Für den Zugriff auf Datenklassen**

Zum Beispiel für den Zugriff auf den Container mit allen verwalteten Objekten oder auf einzelne Datenobjekte zu deren Verwaltung.

- **Für den Zugriff auf grafische Komponenten des Fensters**

Zum Beispiel für den Zugriff auf Textfelder. Das betrifft Komponenten, deren Darstellung sich durch Benutzeraktionen ändert. Um zu entscheiden, ob eine Komponente nur lokal im Konstruktor oder als Attribut definiert werden soll, muss man sich überlegen, wo man auf sie zugreift. Benötigt man den Zugriff auf die Komponente auch außerhalb des Konstruktors, so wird sie als Attribut angelegt, sonst als lokale Variable (oder als anonymes Objekt) im Konstruktor.

- **Zur Verwaltung von internen Zuständen des Fensters**

Das betrifft Zustände, die man für die gewünschte Ereignisbehandlung unterscheiden möchte, zum Beispiel um zu zählen, wie oft der Benutzer auf eine bestimmte Schaltfläche gedrückt hat.

Methoden von Fensterklassen

Im Gegensatz zu Datenklassen implementiert man in Fenstern standardmäßig keine öffentlichen Standardverwaltungsoperationen (also get-, set- und check-Methoden) für dessen Attribute. Solche sieht man nur in Ausnahmen vor, falls man in anderen Klassen Zugriff darauf benötigt.

Stattdessen sieht man private **On-Methoden** vor, in denen man die Reaktion auf Benutzeraktionen kapselt. Gegebenenfalls ergänzt man noch weitere private Hilfsmethoden für bestimmte Teilaufgaben. So wird der Code modularer, besser lesbar und wartbar.

Aktualisierung von Fenstern

In der Fallstudie haben wir den Listen- und den Erfassungsdialog beide als modale Dialoge gewählt.

Stellen Sie sich nun vor, beide Dialoge wären nicht-modal.

In diesem Fall können beide Dialoge gleichzeitig geöffnet sein. Dann würden wir erwarten, dass ein neuer Angestellter im Listendialog sofort angezeigt wird, sobald er über den Erfassungsdialog erstellt wurde. Dazu müsste der Listendialog seine Anzeige aktualisieren, sobald der Container aller Angestellten durch den Erfassungsdialog verändert wurde.

Das können wir zum Beispiel wie folgt bewerkstelligen:

1. **Variante 1 (Datenklassen kennen Fensterklassen, nicht empfohlen)**

Wir tragen in die Klasse `Employee` ein zusätzliches statisches (überlegen Sie) Attribut (zum Beispiel vom Typ `JDialog`) für den Zugriff auf den Listendialog ein: Dieses setzen wir bei Erzeugung des Listendialogs und können den Dialog dann aktualisieren, sobald ein neues `Employee`-Objekt angelegt wurde. Aber nein, das ist keine gute Idee (obwohl es funktioniert): Die Datenklasse sollte keinen vollen Zugriff auf grafische Komponenten haben – das erhöht die Abhängigkeit zwischen den Klassen und verschlechtert die Erweiterbarkeit und Wartbarkeit des Codes. Wir wollen zum Beispiel nicht jedes Mal, wenn wir das Programm um

eine neue grafische Komponente zur Darstellung von Daten erweitern, in der Datenklasse Änderungen vornehmen müssen.

2. **Variante 2 (Fensterklassen kennen sich untereinander, nicht empfohlen)**

Wir könnten stattdessen auf analoge Weise dafür sorgen, dass der Erfassungsdialog (direkt oder indirekt) Zugriff auf den Listendialog hat (überlegen Sie, wie das gehen könnte). Aber auch das erhöht die Abhängigkeit zwischen den Klassen – und jedes Mal, wenn wir das Programm um eine neue grafische Komponente zur Darstellung von Daten erweitern, müssen wir auch in anderen Klassen Änderungen vornehmen.

3. **Variante 3 (Polling, Kapitel 14)**

Eine dritte Möglichkeit wäre, dass der Listendialog regelmäßig in kurzen Zeitabständen (vielfach pro Sekunde) fragt, ob es Änderungen gibt (unabhängig davon, ob sich gerade die darzustellenden Daten geändert haben oder nicht). Dieses Konzept nennt man **Polling**. Dafür muss man einen eigenen asynchronen Prozess erstellen – wie das geht, lernen wir in Kapitel 16 (Threads). Diese Methode ist allerdings in unserem Anwendungsbeispiel, in dem sich die Daten nur in viel größeren Zeitabständen ändern, zu ineffizient.

4. **Variante 4 (Beobachtermuster, Kapitel 11)**

Stattdessen werden wir in Kapitel 11 ein Konzept mit dem Namen **Beobachtermuster** kennenlernen. Das Beobachtermuster kann man (unter anderem) dazu benutzen, um effizient grafische Benutzeroberflächen zu aktualisieren, sobald sich die Daten geändert haben – ohne dass die Datenklassen dazu vollen Zugriff auf grafische Komponenten bekommen. Insbesondere kann man das Modell jederzeit um weitere grafische Komponenten erweitern, ohne dass man nochmal etwas an den Datenklassen (oder anderen bereits existierenden Klassen) ändern muss.

Bis wir die neuen Konzepte kennenlernen, um unser Problem mit der Aktualisierung lösen, geben wir uns mit modalen Dialogen zufrieden (überlegen Sie, warum das geschilderte Problem in diesem Fall nicht auftritt).

8.4.3 Standarddialoge

Abschließend möchten wir noch einen kurzen Überblick darüber geben, wie man in Java einfache Standarddialoge implementieren kann. Dafür hat die Klasse `JOptionPane` verschiedene statische `show`-Methoden zur Erzeugung von

- Eingabedialogen (`showInputDialog`).
- Nachrichtendialogen inklusive Warn- und Fehlerdialogen (`showMessageDialog`).
- Bestätigungsdialogen (`showConfirmDialog`).

Übung: Machen Sie sich damit vertraut, indem Sie zum Beispiel beim Klick des Nutzers auf das Fensterkreuz nicht das Fenster schließen, sondern einen Bestätigungsdialog öffnen, in dem der Nutzer seine Aktion bestätigen soll.

8.5 Zeichnen in Fenstern

Die Implementierung einer Zeichenfläche unterscheidet sich grundsätzlich von den Fenstern der Angestelltenverwaltung. Bei einer Zeichenfläche werden die Daten nämlich nicht über Textfelder (und ähnliche grafische Komponenten wie zum Beispiel Auswahllisten) eingegeben oder angezeigt, sondern über eine Zeichenfläche. Auch für Zeichenflächen stehen grafische API-Komponenten zur Verfügung. Im Unterschied zu den bisherigen grafischen API-Komponenten sorgen wir aber selbst für deren grafischen Inhalt.

8.5.1 Fallstudie: Mausposition zeichnen

Als erstes einfaches Beispiel für eigenes Zeichnen wollen wir eine Zeichenfläche implementieren, in der durchgehend die Mausposition entlang der Mausbewegung (ohne gedrückte Maustaste) als String in der Form (x,y) angezeigt wird. Wir beginnen mit einer UML-Übersicht für die angestrebte Zeichenfläche (Abbildung 24).

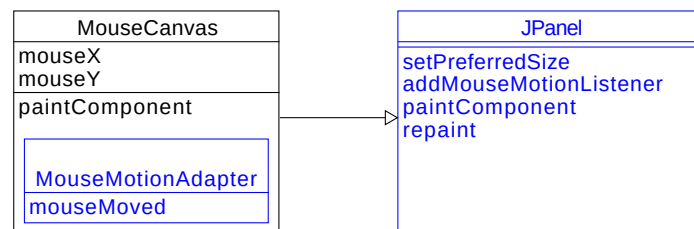


Abbildung 24: UML-Übersicht über eine Zeichenfläche zum Zeichnen der Mausposition.

Wir leiten die Zeichenfläche hier also von einem `JPanel`-Objekt ab. Zudem enthält die Zeichenfläche keine weiteren Komponenten. Solche `JPanel`-Objekte sind deshalb dafür geeignet, da sie keinen eigenen grafischen Inhalt haben. Grundsätzlich kann man auch in nicht-leere grafische Komponenten zeichnen, zum Beispiel auch in eine Schaltfläche, oder auch in Komponenten, die andere Komponenten enthalten. Wenn man aber in eine solche Komponente zeichnet, dann können Konflikte mit anderen grafischen Inhalten im Vordergrund (zum Beispiel Text in Buttons) auftreten und es sind Konflikte mit anderen enthaltenen und vom Layout-Manager positionierten Komponenten möglich.

Für die Daten der Mausposition, die wir zeichnen wollen, legen wir die Attribute `mouseX` und `mouseY` an. Um die jeweilige Mausposition bei Mausbewegungen in diesen Attributen zu speichern, benötigen wir einen Abhörer der Mausbewegung. Diesen werden wir hier als internen Abhörer-Adapter der Zeichenfläche implementieren. Um Mausbewegungen ohne gedrückte Maustaste abzufragen, benötigen wir die `mouseMoved`-Methode.

Schließlich müssen wir noch dafür sorgen, dass die abgehörte Mausposition wie gewünscht entlang der Mausbewegung als Zeichenkette innerhalb der Zeichenfläche gezeichnet wird. Dazu überschreiben wir die `paintComponent`-Methode des `JPanel`-Objekts. Alle leichtgewichtigen Java-Komponenten (wie zum Beispiel `JPanel`) werden nämlich gezeichnet, indem ihre `paint`-Methode

aufgerufen wird. Diese wiederum ruft die `paintComponent`-Methode auf. Diese zeichnet in der Standardimplementierung (nur) den Hintergrund der Komponente. Um zusätzlich noch einen String (oder etwas anderes) zu zeichnen, überschreiben wir diese geeignet.

Die zugehörige Implementierung ist sehr kompakt:

```
public class MouseCanvas extends JPanel {

    private int mouseX;
    private int mouseY;

    public MouseCanvas() {
        this.addMouseListener(new MouseMotionAdapter() {
            public void mouseMoved(MouseEvent e) {
                mouseX = e.getX();
                mouseY = e.getY();
                repaint();
            }
        });
        this.setPreferredSize(new Dimension(500, 500));
    }

    @Override
    protected void paintComponent(Graphics g) {
        g.drawString("(" + mouseX + "," + mouseY + ")", mouseX,
            mouseY);
    }
}
```

In der `mouseMoved`-Methode des Abhörers setzen wir die aktuelle Mausposition und rufen `repaint` auf. Die `repaint`-Methode gehört zur Klasse `Component` und sorgt für das Neuzeichnen einer Komponente durch den Aufruf von `paint`. Durch den Aufruf von `repaint` geschieht das Neuzeichnen in einem sogenannten **asynchronen Prozess**, der keine anderen Teilprozesse und Methoden blockiert (im Gegensatz dazu, wenn man `paint` direkt aufrufen würde).

Mit `paintComponent` können wir in die Zeichenfläche zeichnen. Dazu hat `paintComponent` einen Eingabeparameter `g` vom API-Typ `Graphics`. Dieser Übergabeparameter ist der sogenannte **Grafikkontext** der Zeichenfläche. Dieser definiert eine zweidimensionale Bitmap. Das ist eine Matrix aus Pixeln, für die jeweils eine Position und eine Farbe festgelegt ist. Der Grafikkontext repräsentiert die grafische Darstellung der Zeichenfläche und bietet Methoden zum Festlegen von Zeichenfarbe und Schriftart und zum Zeichnen verschiedener Inhalte wie Strings, Ellipsen, Rechtecken, Linien und Polygonen. Zum Beispiel können wir mit `g.drawString(...)` einen String (erster Eingabeparameter) an eine übergebene Mausposition (zweiter und dritter Eingabeparameter) zeichnen. Alte Zeichnungen werden bei jeder Mausbewegung überschrieben, weil jeweils automatisch der Hintergrund neu gezeichnet wird.

Um die Zeichenfläche testen zu können, müssen wir diese nun nur noch einem Fenster hinzufügen. Beachten Sie, dass es hier, wie in der Fallstudie `ClickFrame` in Unterkapitel 8.3.1, unterschiedliche mögliche Implementierungen des Abhörers gibt.

Übung: Implementieren Sie die Zeichenfläche zur Übung in mindestens 2 weiteren verschiedenen Varianten.

8.5.2 Allgemeines Konzept

Nach diesem kleinen Einführungsbeispiel wollen wir uns einen systematischen Überblick darüber verschaffen, wie in Java grafische Komponenten gezeichnet werden. Dazu rufen wir uns die Vererbungshierarchie aller grafischen Komponenten aus Abbildung 5 in Erinnerung. Die gemeinsame Oberklasse `Component` stellt verschiedene Methoden zum Zeichnen einer Komponente zur Verfügung.

Definition 8.12: Zeichnen einer Komponente (Klasse `Component`)

Auswahl einiger Methoden zum Zeichnen eines `Component`-Objekts:

- `public Graphics getGraphics()`
Liefert eine Repräsentation der grafischen Informationen einer Komponente als `Graphics`-Objekt.
- `public void paint(Graphics g)`
Zeichnet eine Komponente komplett neu (unterschiedliche Implementierung in Unterklassen).
- `public void repaint()`
Startet einen asynchronen Prozess zum Neuzeichnen einer Komponente.

Jede grafisch dargestellte Komponente besitzt dazu einen sogenannten **Grafikkontext** vom Typ `Graphics`. Der Grafikkontext definiert eine **zweidimensionale Bitmap** zur Repräsentation der grafischen Darstellung. Darüber hinaus enthält er Angaben über den relevanten **Bildausschnitt**, sowie die **Zeichenfarbe** und die **Schriftart** für neue Zeichnungen. Die Klasse stellt eine Reihe von **Methoden zum Zeichnen** zur Verfügung.

Definition 8.13: Zeichnen in eine Komponente (Klasse Graphics)

Über den Grafikkontext einer Komponente kann man in die Komponente zeichnen. Dafür stehen eine Reihe von `Graphics`-Methoden zur Verfügung, zum Beispiel:

- `drawOval(int x, int y, int width, int height)`
Eine Ellipse an Position `(x,y)` zeichnen mit Höhe `height` und Breite `width`.
- `setColor(Color c)`
Die Farbe für die nächste Zeichnung setzen.
- `drawRect(int x, int y, int width, int height)`
Ein Rechteck an Position `(x,y)` zeichnen mit Höhe `height` und Breite `width`.
- `drawString(String str, int x, int y)`
Den String `str` an Position `(x,y)` zeichnen.

Die Position ist dabei jeweils die linke obere Ecke der Zeichnung relativ zur linken oberen Ecke der Komponente.

Die Klasse `Graphics` und relevante Unterklassen sind dabei **abstrakt**, sodass `Graphics`-Objekte nicht direkt erzeugt werden können. Die Komponentenverwaltung des Java-Laufzeitsystems erzeugt `Graphics`-Objekte mittels **interner Klassen** und ordnet sie Komponenten zu. Eine Komponente kann mit Hilfe der `getGraphics`-Methode ihren Grafikkontext zur Verfügung stellen. Dabei wird ein Objekt einer nicht-abstrakten Unterklasse von `Graphics` zurückgeliefert.

Das zu einer Komponente gehörende `Graphics`-Objekt wird zudem an die `paint`-Methode übergeben. Diese sorgt dafür, dass die Komponente gezeichnet wird und ist in verschiedenen Unterklassen unterschiedlich implementiert. Für diese Vorlesung interessieren uns hier vor allem schwergewichtige Swing-Komponenten (wie `JFrame` und `JDialog`) und leichtgewichtige Swing-Komponenten (also alle Unterklassen von `JComponent` wie zum Beispiel `JPanel`).

Definition 8.14: Zeichnen schwergewichtiger Swing-Komponenten

Die Standardimplementierung von `paint` eines schwergewichtigen Swing-Fensters zeichnet Rand, Titel und Hintergrund des Fensters.

Zudem werden die im Fenster enthaltenen leichtgewichtigen Komponenten gezeichnet, indem jeweils deren `paint`-Methode aufgerufen wird.

Die Standardimplementierung von `paint` lässt sich in einem eigenen zum Beispiel von `JFrame` abgeleiteten Fenster überschreiben, um eigene Zeichnungen anzuzeigen. So kann man mit Mausergebnissen direkt in ein Fenster zeichnen, ohne dass man wie in der vorigen Fallstudie eine Zeichenfläche benutzt. Die überschriebene `paint`-Methode würde dann wie folgt aussehen:

```
public void paint(Graphics g) {  
    super.paint(g);  
    g.drawString("(" + mouseX + "," + mouseY + ")", mouseX, mouseY);  
}
```

Bei der Implementierung der `paint`-Methode eines Fensters verwenden wir grundsätzlich folgende Strategie:

Wir rufen zuerst mit `super.paint(g)` die `paint`-Implementierung der Oberklasse auf

Dadurch wird der Hintergrund neu gezeichnet und damit werden alle bisherigen Zeichnungen (alte Mauspositionen) überschrieben, so dass immer nur die aktuelle Mausposition angezeigt wird. Beachten Sie, dass ein entsprechendes Vorgehen bei `JPanel` nicht notwendig war.

Übung: Implementieren Sie diese Variante, und testen Sie den Effekt, wenn man die Oberklassenimplementierung nicht aufruft.

Die Verwendung von schwergewichtigen Fenstern zum Zeichnen hat Nachteile:

- Bei häufigem Neuzeichnen kann es zu Verzögerungen (Flackern) kommen, da die Zwischenpufferung der zu zeichnenden Inhalte für schwergewichtige Fenster nicht so effizient implementiert ist wie für leichtgewichtige Komponenten (Fenster sind nicht für häufiges Neuzeichnen ausgelegt).
- Das Fenster ist nicht als Zeichenfläche in anderen Anwendungen wiederverwendbar.

Besser benutzt man deshalb wie in der Fallstudie eine leichtgewichtige Komponente als Zeichenfläche.

Die `paint`-Methode leichtgewichtiger Swing-Komponenten hat eine andere Standardimplementierung als diejenige von Swing-Fenstern. Dazu rufen wir uns kurz aus Unterkapitel 7.1 in Erinnerung, dass jede leichtgewichtige Swing-Komponente ein Container-Objekt ist und nur innerhalb eines schwergewichtigen Fensters angezeigt werden kann:

- Eine leichtgewichtige Swing-Komponente ist also immer selbst Teil eines anderen Container-Objekts. Dieses nennen wir ihre **Elternkomponente**. Zum Beispiel kann ein Fenster über mehrere ineinander verschachtelte Container-Objekte hierarchisch gegliedert sein. So kann ein `JButton`-Objekt Teil eines `JPanel`-Objekts sein, welches wiederum Teil eines `JFrame`-Objekts ist. Dann ist das `JFrame`-Objekt die Elternkomponente vom `JPanel`-Objekt, welches wiederum die Elternkomponente vom `JButton`-Objekt ist.
- Umgekehrt nennen wir alle in einem Container-Objekt enthaltenen Komponenten dessen **Kindkomponenten**.

Außerdem spielt es für das Zeichnen einer leichtgewichtiger Swing-Komponenten eine Rolle, ob diese **durchsichtig** oder **undurchsichtig** ist. Dazu hat jede solche Komponente eine Standardinstellung, die mit der `setOpaque`-Methode verändert werden kann. Bei einer undurchsichtigen Komponente ist sichergestellt, dass stets alle ihre Pixel neu gezeichnet werden und damit keine Pixel der Elternkomponente sichtbar sind. Bei durchsichtigen Komponenten hingegen können Pixel der Elternkomponente sichtbar sein (so werden zum Beispiel runde Ecken von eigentlich rechteckigen Buttons hergestellt – ein solcher Button ist in den Ecken transparent). Wird die Darstellung einer durchsichtigen Komponente aktualisiert, muss deshalb immer auch die Elternkomponente mit aktualisiert werden.

Definition 8.15: Zeichnen leichtgewichtiger Swing-Komponenten

Die Standardimplementierung von `paint` einer leichtgewichtigen Swing-Komponente ruft nacheinander in der angegebenen Reihenfolge folgende Methoden auf:

1. `public void paintComponent(Graphics g)`
Zeichnet den Hintergrund, falls die Komponente undurchsichtig ist. Falls sie durchsichtig (transparent) ist, wird die Elternkomponente automatisch mit aktualisiert. Da das Zeichnen in diesem Fall deutlich ineffizienter ist, benutzen wir immer wenn möglich undurchsichtige Komponenten.
2. `public void paintBorder(Graphics g)`
Zeichnet den Rand (falls vorhanden).
3. `public void paintChildren(Graphics g)`
Zeichnet alle enthaltenen Kindkomponenten (falls vorhanden).

Auch diese Standardimplementierung lässt sich in einer eigenen abgeleiteten Komponente überschreiben, um eigene Zeichnungen anzuzeigen. Dazu überschreibt man allerdings nicht `paint` sondern `paintComponent`, so dass das Zeichnen des Randes und der Kindkomponenten nicht beeinträchtigt wird. Auf diese Weise können wir zum Beispiel wie in der vorigen Fallstudie eine eigene Zeichenfläche implementieren.

Wir gehen hier also ähnlich wie bei schwergewichtigen Fenstern vor, überschreiben nur eine andere Methode. Grundsätzlich kann man übrigens auch in standardmäßig nicht leere Komponenten zeichnen. Dann können jedoch Konflikte mit anderen grafischen Inhalten im Vordergrund (zum Beispiel Text in Buttons) auftreten und es sind Konflikte mit anderen vom Layout-Manager positionierten Kindkomponenten möglich.

Definition 8.16: In eine Swing-Komponente zeichnen

Für eigene Zeichenflächen eignen sich standardmäßig leere Swing-Komponenten ohne Rand und ohne grafischen Inhalt wie zum Beispiel `JPanel` (siehe Eingangsbeispiel) oder `JComponent`. Dabei sollten eigene Zeichenflächen undurchsichtig sein (überlegen Sie).

Für eigene Zeichnungen überschreiben wir geeignet die `paintComponent`-Methode. Wollen wir dabei auch Zeichnungen der Oberklasse erhalten, rufen wir als erste Anweisung die Implementierung der Oberklasse auf (`super.paintComponent(g)`).

Es gibt nun verschiedene Ursachen dafür, dass eine Komponente durch Aufruf der `paint`-Methode neu gezeichnet wird. Der Auslöser für einen internen Aufruf von `paint` kann entweder eine Programmanweisung sein (zum Beispiel ein Aufruf von `setVisible`) oder auch eine Anweisung des Betriebssystems (zum Beispiel, wenn ein Fenster ein anderes Fenster verdeckt und dann geschlossen oder verschoben wird). Dabei wird nie direkt `paint` aufgerufen, denn `paint` ist eine sogenannte

synchrone Methode: Ihre Abbarbeitung blockiert alle anderen Methoden – während das Fenster gezeichnet wird, würden andere Methoden „hängen bleiben“. Stattdessen wird intern immer `repaint` aufgerufen, wenn eine Komponente gezeichnet werden soll. Die `repaint`-Methode ist eine sogenannte **asynchrone Methode:** Ihre Verarbeitung erfolgt in einem sogenannten **asynchronen Prozess** (der den Aufruf von `paint` enthält), der parallel (zeitgleich) zu allen anderen Methoden ablaufen kann. In Kapitel 14 werden wir asynchrone Prozesse genauer betrachten und lernen, wie man als Programmierer eigene asynchrone Prozesse definieren kann.

Definition 8.17: Systemgesteuertes Zeichnen

Vom **systemgesteuerten Zeichnen** spricht man, wenn das System das Neuzeichnen einer Komponente anfordert, zum Beispiel wenn

- eine Komponente das erste Mal gezeichnet wird.
- sich die Größe einer Komponente ändert.
- eine Komponente (teilweise) durch eine andere Komponente verdeckt war.

Die Durchführung des systemgesteuerten Zeichnens startet grundsätzlich mit einem schwergewichtigen Fenster. Dazu wird dessen `paint`-Methode aufgerufen. Diese sorgt dann wie beschrieben auch für das Zeichnen aller enthaltenen Komponenten. Damit der Zeichenvorgang keine andere Prozesse und Berechnungen blockiert, findet er in einem asynchronen Prozess statt (Aufruf von `repaint`).

Definition 8.18: Anwendungsgesteuertes Zeichnen

Vom **anwendungsgesteuerten Zeichnen** spricht man, wenn der Programmierer durch Aufruf einer Methode das Neuzeichnen einer Komponente anfordert. Dies sollte **grundsätzlich** wiederum durch Aufruf der `repaint`-Methode der Komponente geschehen, da diese **asynchron** ist und so nicht die Anwendung blockiert. Deshalb sollte man **auf keinen Fall** für das anwendungsgesteuerte Zeichnen die `paint`-Methode verwenden!

Die Implementierung von `repaint` hängt von der Art der Komponente ab. Bei allen Swing-Komponenten und -fenstern ruft `repaint` direkt oder indirekt `paint` auf. Für Swing-Komponenten und -fenster gibt es damit tatsächlich keinen grundsätzlichen Unterschied zwischen system- und anwendungsgesteuertem Zeichnen, außer dass auch einzelne Komponenten anwendungsgesteuert neu gezeichnet werden können. Bei AWT-Komponenten verhält sich dies anders.

Weitere Details sind auf der Seite <https://www.oracle.com/java/technologies/painting.html> beschrieben.

8.5.3 Fallstudie: Grafischer Editor

Damit haben wir alle Voraussetzungen besprochen, um die Benutzeroberfläche des grafische Editors zu implementieren. Dieser soll am Ende etwa wie in Abbildung 25 aussehen. Diese Abbildung

zeigt das Hauptanwendungsfenster, das folgende grafische Komponenten enthält:

- Eine Zeichenfläche im Zentrum. In diese soll der Benutzer mit Mauseaktionen Kreise und Rechtecke zeichnen können. Das soll einmal möglich sein per Mausklick (Erzeugung einer Form mit fester Standardgröße). Zweitens soll der Benutzer mit gedrückter Maustaste eine Form aufziehen können.
- Einem Panel mit vier Buttons im Norden. Die Buttons sind dazu da, eine Vorauswahl für Benutzeraktionen mit der Maus in der Zeichenfläche zu treffen. Klickt der Benutzer zum Beispiel auf `Draw circle`, soll der Titel entsprechend anzeigen, dass der Benutzer mit dem nächsten Mausklick einen Kreis fester Größe in die Zeichenfläche zeichnen kann. Zudem soll die Zeichenfläche auf den nächsten Mausklick genau so reagieren. Dazu hört die Zeichenfläche die Buttons ab und merkt sich die Vorauswahl des Benutzers in boolschen Variablen.

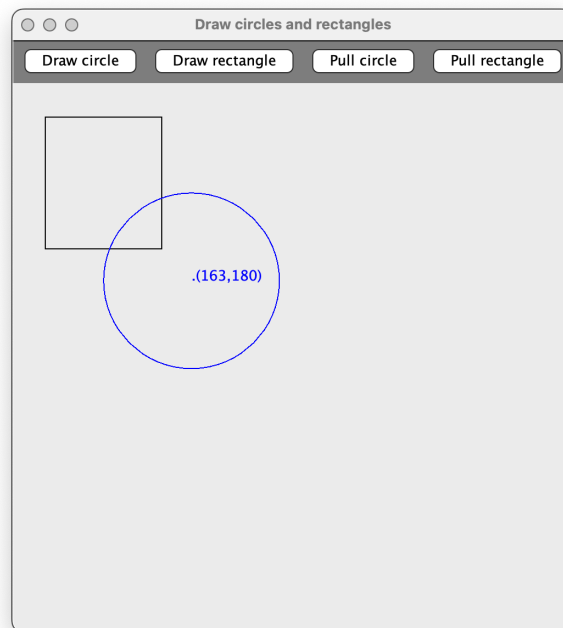


Abbildung 25: Screenshot des Editors.

Unser Plan zur Implementierung des gezeigten Hauptanwendungsfensters in UML sieht wie in Abbildung 26 aus:

- Das Hauptanwendungsfenster hört die Buttons ab um den Titel entsprechend der Vorauswahl zu setzen. Drückt der Benutzer zum Beispiel aus den Button `"Draw circle"`, so soll dieser String danach im Titel angezeigt werden, damit klar ist, dass mit dem nächsten Mausklick ein Kreis gezeichnet wird.
- Ebenso hört die Zeichenfläche `canvas` die Buttons ab, damit diese bei Mouseevents korrekt aktualisiert wird. Drückt der Benutzer zum Beispiel aus den Button `"Draw circle"`, so werden wir in der Zeichenfläche die boolsche Variable `isDrawCircle` auf `true` setzen (siehe auch nachfolgende Beschreibung zur Zeichenfläche). Beim nächsten Mausklick in die Zeichenfläche werden wir diese boolsche Variable abfragen, um dann den Kreis an der Klickposition zu zeichnen.

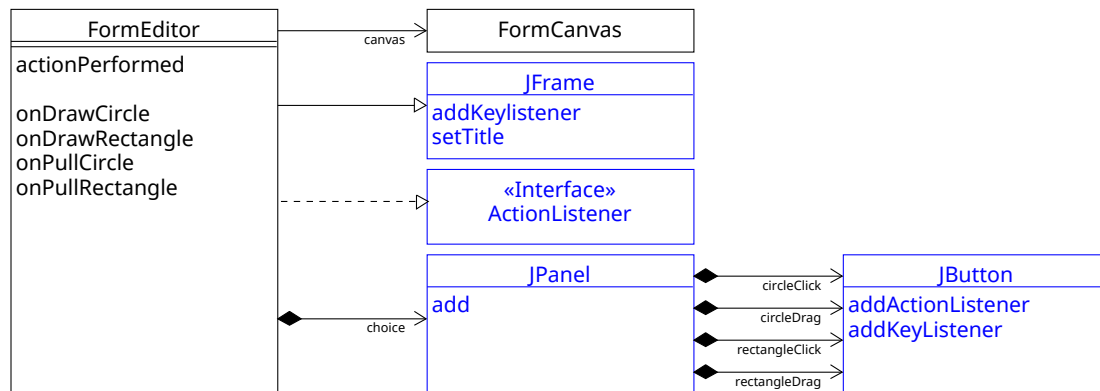


Abbildung 26: UML-Entwurf des Hauptanwendungsfensters des Editors.

- Ein besonderes Augenmerk müssen wir auf Keyevents legen. Diese soll die Zeichenfläche abhören, da wir Formen mit den Pfeiltasten bewegen wollen. Bei einem Keyevent ist immer die Komponente die Ereignisquelle, die bei Drücken einer Taste gerade den sogenannten **Fokus** hat. Das kann zum jeweiligen Zeitpunkt das Fenster selbst und jede der Komponenten des Fensters (also eine der Buttons) sein. Alle diese Komponenten müssen also von der Zeichenfläche abgehört werden.

Unser Plan zur Implementierung der Zeichenfläche in UML sieht wie in Abbildung 27 aus. Wie schon bei Fenstern und Buttons diskutiert, implementieren wir auch eine Zeichenfläche als Unterklasse einer geeigneten API-Komponente (hier: `JPanel`). Dadurch stehen uns alle Grundfunktionalitäten der API-Komponente zur Verfügung und wir kapseln Informationen, Aussehen und Verhalten übersichtlich wartbar in einer Klasse. Außerdem macht man die Zeichenfläche wiederverwendbar.

Man sieht, dass die Zeichenfläche selbst alle auftretenden Ereignisse abhört. Das ist deshalb sinnvoll, da als Reaktion auf Benutzeraktionen jeweils etwas in der Zeichenfläche gezeichnet werden soll. Für Mouseevents ist die Zeichenfläche selbst auch die Ereignisquelle – die Zeichenfläche hört sich hier also selbst ab.

Von den angebundenen API-Komponenten ist nur das Popuptmenü neu. Dieses funktioniert wie eine normales Menü. Menüeinträge des Popuptmenüs erzeugen also Actionevents. Über Menüeinträge wollen wir die gleichen Vorauswahlen erlauben wie über die Buttons des Hautanwendungsfensters. Zudem sollen über einen Menüeintrag ausgewählte Formen gelöscht werden können.

Die Zeichenfläche hat mehrere Attribute:

- Mit den boolschen Is-Attributen wollen wir uns die Vorauswahl des Benutzers durch Klicks auf Buttons oder Menüeinträge merken. Zum Beispiel werden wir `isDrawCircle` auf `true` setzen, wenn der Benutzer auf den Button (oder Menüeintrag) "Draw circle" drückt.
- Über die beiden `Position`-Attribute werden wir uns bestimmte Mauspositionen merken: In `mousePressedPosition` die Drückposition, bei der das Aufziehen einer Form beginnt, und in `mouseOldPosition` die Drückposition, bei der die Bewegung einer Form beginnt.

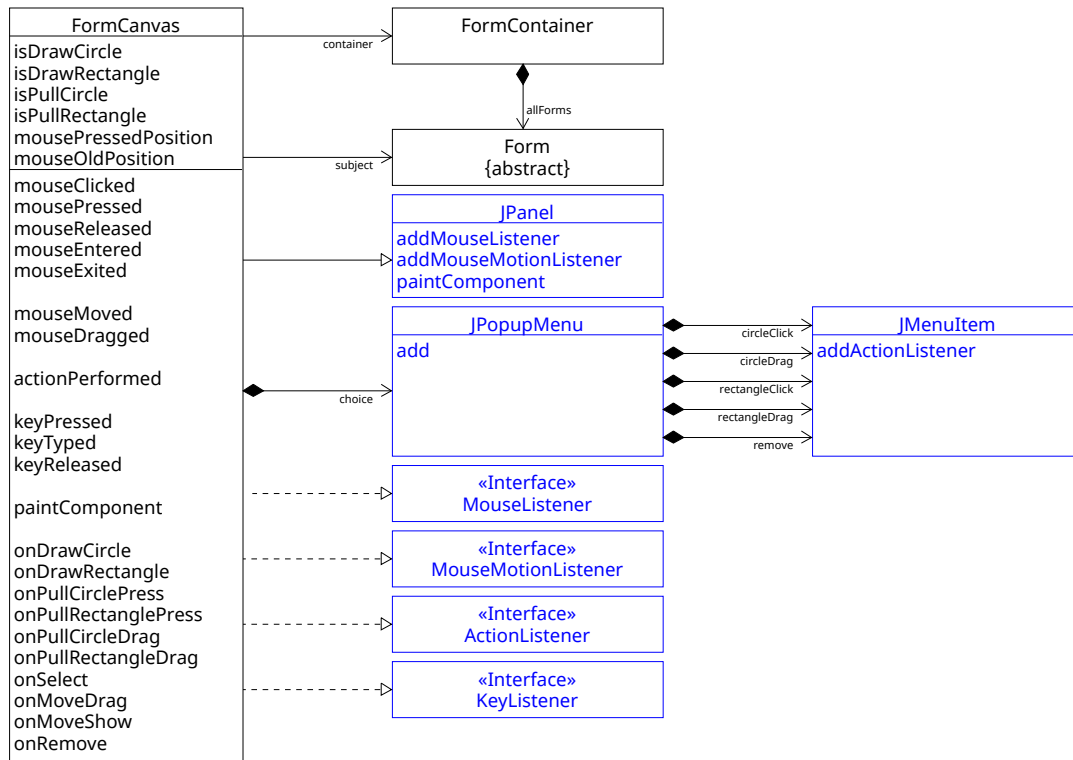


Abbildung 27: UML-Entwurf der Zeichenfläche des Editors.

Neben den Abhörermethoden sehen wir die Methode `paintComponent` zum Zeichnen und viele `On`-Methoden für die Reaktion auf Benutzeraktionen.

Die Zeichenfläche hat neben einem Referenzattribut `container` zur Anbindung des Datencontainers noch ein Referenzattribut `subject` für den Zugriff auf eine neu erfasste Form. Wir nehmen das Popup-Menü als Referenzattribut auf, da dieses in der Mausab Hörer-Methode `mousePressed` erzeugt wird. Zudem ist der Editor selbst ein Referenzattribut, da er Ereignisse des Popup-Menüs abhört.

Vier Einträge im Popup-Menü heißen genauso wie die Buttons des Editors. Damit kann die Vorauswahl für die nächste Benutzeraktion auch über das Pop-upmenü erfolgen. Dafür sind sowohl die Zeichenfläche also auch der Editor Abhörer:

- In der Zeichenfläche werden durch die Vorauswahl interne Attribute gesetzt und so die Reaktion auf nachfolgende Mausereignisse gesteuert.
- Im Editor wird der Titel an die Vorauswahl angepasst.

Zudem können über den Menü-Eintrag "Remove selected forms" ausgewählte Formen gelöscht werden. Dieses Actionevent hört nur die Zeichenfläche ab.

```

public class FormCanvas extends JPanel implements ActionListener,
    MouseListener, MouseMotionListener, KeyListener {

```

```

private Form subject = null;
private FormContainer container = new FormContainer();
private JPopupMenu choice;
private FormEditor editor;
...

public FormCanvas(FormEditor editor) {
    this.editor = editor;
    ...

    choice = new JPopupMenu();
    JMenuItem circleClick = new JMenuItem("Draw circle");
    circleClick.addActionListener(this);
    circleClick.addActionListener(editor);
    choice.add(circleClick);

    JMenuItem remove = new JMenuItem("Remove selected forms");
    remove.addActionListener(this);
    choice.add(remove);
    ...
}
...
}

```

Wir beschreiben exemplarisch einige der Methoden mit ihrer Implementierung:

Implementierung von paintComponent

```

@Override
public void paintComponent(Graphics g) {
    for (Form o : container)
        o.paint(g);
}

```

Zum Zeichnen aller Formen wird der Container durchlaufen, und für jede Form deren paint-Methode aufgerufen. Dafür wird der Grafikkontext der Zeichenfläche an die paint-Methode der Form durchgereicht. Die Formen zeichnen sich also selbst.

Diese Zuordnung macht Sinn, denn die Zeichnung hängt direkt von den Daten (Position und Dimensionen) einer Form ab.

So fassen wir zusammengehörigen Informationen und Funktionalitäten an einem Ort zusammen, was das Programm übersichtlicher und besser wartbar macht. In der Klasse Circle sieht die Implementierung der paint-Methode beispielsweise wie folgt aus:

```

public void paint(Graphics g) {
    g.setColor(this.getBorderColor());
    g.drawOval(getPosition().getX() - radius, getPosition().getY() -
        radius, 2 * radius, 2 * radius);
    if (isSelected())

```

```

        g.drawString(".", getPosition().toString(), getPosition().
            getX(), getPosition().getY());
    }

```

Zuerst wird die Farbe der Form für den Grafikkontext gesetzt (`setColor`). Danach wird der Kreis gezeichnet (`drawOval`). Dabei ist zu beachten, dass die Position einer Form deren Mittelpunkt ist, `drawOval` aber als Positionsangabe die linke obere Ecke (des Rechtecks, in dem der Kreis eingeschrieben ist) erwartet. Die ersten beiden Parameter bestimmen diese Position. Die letzten beiden Parameter bestimmen Breite und Höhe des Rechtecks, in das der Kreis eingeschrieben ist (beide entsprechen dem Durchmesser des Kreises). Schließlich wird bei ausgewählten Formen noch die Position als String angezeigt (`drawString`).

Implementierung von `actionPerformed`

```

public void actionPerformed(ActionEvent e) {
    if (e.getActionCommand().equals("Draw circle")) {
        isDrawCircle = true;
    }
    else if (e.getActionCommand().equals("Draw rectangle")) {
        isDrawRectangle = true;
    }
    ...
    else if (e.getActionCommand().equals("Remove selected forms")) {
        onRemove();
    }
}

```

Über die Schaltflächen des Fensters oder die Menüeinträge des Popupmenüs wählt der Benutzer aus, welche Aktion er als nächstes ausführen möchte. Die Auswahl merken wir uns für Vorgänge zum Zeichnen in boolschen Variablen, da hierfür dann weitere Ereignisse abgehört werden müssen. Nur das Löschen von Formen können wir direkt ausführen.

Implementierung von `mouseClicked`

```

public void mouseClicked(MouseEvent e) {
    if (isDrawCircle == true)
        onDrawCircle(e);
    else if (isDrawRectangle == true)
        onDrawRectangle(e);
    else {
        onSelect(e);
    }
}

```

Klickt der Benutzer in die Zeichenfläche, hängt es von der vorausgewählten Aktion ab, was dann passieren soll: Entweder wird ein Kreis oder ein Rechteck gezeichnet, oder es werden Formen per Mausklick ausgewählt. Dabei wird das Mousevent jeweils weitergereicht, um die Klickposition abfragen zu können.

Je nach Aktions-Vorauswahl muss die Reaktion der Zeichenfläche auf ein Mausereignis also unterschiedlich ausfallen

Dieses Prinzip betrifft auch alle anderen Mausereignisse.

Implementierung von onDrawCircle

```
private void onDrawCircle(MouseEvent e) {
    subject = new Circle(new Point(e.getX(), e.getY()));
    container.addForm(subject);
    repaint();
    clearAction();
    clearTitle();
}
```

Hier wird nun mit einem Konstruktoraufbau ein Kreis fester Größe an der Mausposition (`e.getX()` und `e.getY()`) erstellt und in den Container aufgenommen. Danach wird die Zeichenfläche aktualisiert, damit die neu in den Container aufgenommene Form auch gezeichnet wird (`repaint()`). Die Methode `clearAction` ist eine Hilfsmethode, die alle Variablen zur Aktions-Vorauswahl wieder auf `false` setzt. Damit ist die Vorauswahl immer nur für einen Mausklick gültig. Die Methode `clearTitle` ist eine Hilfsmethode, die den Titel des Editors zurücksetzt.

Implementierung von onSelect

```
private void onSelect(MouseEvent e) {
    for (Form o : container) {
        if (o.mouseOver(new Point(e.getX(), e.getY())))
            o.setSelected(true);
        else
            o.setSelected(false);
        repaint();
    }
}
```

Hat der Benutzer keine Aktions-Vorauswahl getroffen, sollen mit einem Mausklick diejenigen Formen ausgewählt werden, in denen sich die Mausposition befindet. Dazu werden alle Formen durchlaufen und jeweils mit `mouseOver` überprüft, ob die Mausposition innerhalb der Form ist. Danach wird die Zeichenfläche aktualisiert, damit die geänderte Farbe der ausgewählten Formen gezeichnet wird.

Implementierung von mousePressed

```
public void mousePressed(MouseEvent e) {
    if (e.isPopupTrigger()) {
        choice.show(e.getComponent(), e.getX(), e.getY());
    } else if (pullCircle == true) {
        onPullCirclePress(e);
    }
}
```

```

    } else if (pullRectangle == true) {
        onPullRectanglePress(e);
    } else {
        mouseOldPosition = new Point(e.getX(), e.getY());
        onSelect(e);
    }
}

```

Drückt der Benutzer die Maustaste, hängt es von der Art der Maustaste und vorausgewählten Aktion ab, was dann passieren soll: Entweder wird mit der rechten Maustaste das Popupmenü angezeigt, oder es wird nach Vorauswahl ein Kreis oder ein Rechteck mit gedrückter Maustaste aufgezogen, oder es sollen ausgewählte Formen mit gedrückter Maustaste verschoben werden. In letzterem Fall merken wir uns die Mausposition als alte Position `mouseOldPosition` für Mausebewegungen (siehe auch `onMoveDrag`).

Implementierung von `onPullCirclePress`

```

private void onPullCirclePress(MouseEvent e) {
    mousePressedPosition = new Point(e.getX(), e.getY());
    subject = new Circle(0, new Point(mousePressedPosition));
    container.addForm(subject);
    repaint();
}

```

Hat der Benutzer die Vorauswahl getroffen, einen Kreis aufzuziehen, und drückt der Benutzer die Maustaste, so erzeugen wir an der Drückposition einen Kreis mit Radius 0 (wieso erzeugen wir für die Kreisposition eine Kopie der Drückposition?). Wenn der Benutzer danach mit gedrückter Maustaste die Maus bewegt (`mouseDragged`), passen wir den Radius des Kreises über die Referenz `subject` entlang der Mausebewegung so an, dass der Radius immer das Minimum des Abstands der Maus von der Drückposition in X- und in Y-Richtung ist (`onPullCircleDrag`). Dazu merken wir uns die Drückposition in `mousePressedPosition`.

Implementierung von `mouseDragged`

```

public void mouseDragged(MouseEvent e) {
    if (isPullCircle == true) {
        onPullCircleDrag(e);
    } else if (isPullRectangle == true) {
        onPullRectangleDrag(e);
    } else {
        onMoveDrag(e);
    }
}

```

Bewegt der Benutzer die Maus mit gedrückter Maustaste, so wird in regelmäßigen sehr kurzen Zeitabständen ein Mausereignis erzeugt und `mouseDragged` aufgerufen. Dann hängt es von der vorausgewählten Aktion ab, was passieren soll: Entweder wird ein Kreis oder ein Rechteck aufgezogen, oder es werden ausgewählte Formen verschoben.

Implementierung von onPullCircleDrag

```
private void onPullCircleDrag(MouseEvent e) {
    int x = Math.max(0, e.getX());
    int y = Math.max(0, e.getY());
    int r = (int) Math.min(Math.abs(x - mousePressedPosition.getX()),
        Math.abs(y - mousePressedPosition.getY()));
    ((Circle) subject).setRadius(r);
    repaint();
}
```

Hat der Benutzer die Vorauswahl getroffen, einen Kreis aufzuziehen, und bewegt der Benutzer die Maus mit gedrückter Maustaste, so passen wir den Radius des Kreises über die Referenz subject entlang der Mausbewegung so an, dass der Radius immer das Minimum des Abstands der Maus von der Drückposition in X- und in Y-Richtung ist. Dabei begrenzen wir Mauspositionen auf nicht-negative Werte.

Implementierung von onMoveDrag

```
private void onMoveDrag(MouseEvent e) {
    int dx = e.getX() - mouseOldPosition.getX();
    int dy = e.getY() - mouseOldPosition.getY();
    for (Form o : container) {
        if (o.isSelected()) {
            o.move(dx, dy);
        }
        repaint();
    }
    int x = Math.max(0, e.getX());
    int y = Math.max(0, e.getY());
    mouseOldPosition = new Point(x, y);
}
```

Sollen ausgewählte Formen verschoben werden, wird zuerst der Abstand der aktuellen Mausposition zur alten Mausposition entlang der Mausbewegung berechnet (dx und dy). Dann werden alle Formen im Container durchlaufen, und jede ausgewählte Form (o.isSelected()) um diesen Abstand verschoben (o.move(dx, dy)). Danach ist die aktuelle Mausposition die neue alte Mausposition für die Mausbewegung, wobei wir keinen negativen Mauspositionen erlauben.

Implementierung von mouseReleased

```
public void mouseReleased(MouseEvent e) {
    if (isPullCircle || isPullRectangle) {
        clearAction();
        clearTitle();
    }
    for (Form o : container) {
```

```

        o.setSelected(false);
    }
    repaint();
}

```

Lässt der Benutzer die Maustaste nach dem Aufziehen einer Form los, so wird die Vorauswahl zurückgesetzt, da diese nur für eine Benutzeraktion gültig sein soll. Lässt der Benutzer die Maustaste nach dem Bewegen von Formen los, so wird die Auswahl der Formen aufgehoben.

Implementierung von onRemove

```

private void onRemove() {
    Form toRemove = null;
    for (Form o : container) {
        if (o.isSelected()) {
            toRemove = o;
            break;
        }
    }
    if (toRemove != null) {
        container.removeForm(toRemove);
        repaint();
        onRemove();
    }
}

```

Wählt der Benutzer im Popupmenü aus, ausgewählte Formen zu löschen, so wird der Container durchlaufen, und jede ausgewählte Form mit `removeForm` aus dem Container entfernt. Die Methode ist dabei rekursiv so implementiert, dass alle ausgewählten Formen einzeln nacheinander gelöscht werden. Ansonsten kann es bei der Anwendung des Iterators zu einer sogenannten `ConcurrentModificationException` kommen (Eigenrecherche).

Übung

Alle nicht gezeigten Aspekte der Implementierung überlassen wir Ihnen zu Übung. Dazu gehört insbesondere das Hauptanwendungsfenster. Zudem fehlen noch die folgenden Methoden der Zeichenfläche:

- `onDrawRectangle`:
Erzeugt ein neues Rechteck mit Standardgröße und nimmt es in den Container auf.
- `onPullRectanglePress`:
Merkt sich die Startposition zum Aufziehen, erzeugt ein neues Rechteck mit Breite und Länge 0 und nimmt es in den Container auf.
- `onPullRectangleDrag`:
Passt Breite und Länge des Rechtecks `subject` an die Mausbewegung an (durch den Abstand der aktuellen Mausposition von der Startposition für das Aufziehen von `subject`).

- `mouseMoved` und `onMoveShow`:
Bewegt der Benutzer die Maus ohne gedrückte Maustaste, sollen nicht ausgewählte Formen, in denen sich die Maus befindet, in einer anderen Farbe angezeigt werden.
- `keyPressed`:
Durch Drücken auf Pfeiltasten sollen ausgewählte Formen bewegt werden.

Bringen Sie den kompletten Editor nun selbstständig zum Laufen!

Wenn Sie zudem Lust haben, können Sie das Programm auch noch um weitere sinnvolle Funktionalitäten erweitern, zum Beispiel:

- Kopieren von Formen.
- Verändern der Größe von Formen.
- Mehrere Formen mit gedrückter Shifttaste + Mausklick auswählen.

Das notwendige Handwerkszeug dafür haben Sie nun zur Verfügung.

8.6 Weitere Fallstudien

Ein weiteres Beispiel wird im Screencast *SC-Ein-sehnsüchtiger-Blick-aus-dem-Fenster-Fensterklassen-und-Zeichnen* besprochen.